



Structural Health Monitoring using Physics-Informed Neural Networks for Multi-Degree-of-Freedom System Modelling and Parameter Identification

Project Report

Supervised by:

Prof. Dr. Tom Lahmer

Rohan Raj Das

Submitted by:

Saggu Nagaraju 126794

M.Sc Digital Engineering

April 2026

Contents

1	Introduction	8
1.1	Structural Health Monitoring (SHM)	8
1.2	Importance of Structural Health Monitoring	9
1.3	Why Multi-Degree-of-Freedom (MDOF) Systems Are Used	10
1.4	Why Physics-Informed Neural Networks (PINNs) Are Useful	10
1.5	Research Objective	11
1.6	Scope of the Work	12
2	Structural Health Monitoring and Structural Dynamics Fundamentals	13
2.1	Structural Dynamics	13
2.2	Single Degree of Freedom (SDOF) System	13
2.2.1	Basic Components of an SDOF System	13
2.2.2	Equation of Motion of an SDOF System	14
2.2.3	Natural Frequency	15
2.2.4	Damping Ratio	15
2.3	Multi Degree of Freedom (MDOF) Systems	16
2.3.1	Concept of MDOF Systems	16
2.3.2	Equation of Motion of MDOF System	17
2.3.3	Modal Properties	17
2.4	Structural Health Monitoring Concepts	20
2.5	Physics-Informed Neural Networks (PINNs)	21
2.5.1	Data Loss	22
2.5.2	Physics Loss	22
2.5.3	Total Loss Function	22
2.6	Importance of MDOF Models in Structural Health Monitoring	23
2.7	Connection to the Present Project	23
3	Mathematical Formulation	25
3.1	Governing Equations of Structural Dynamics	25
3.2	Direct Displacement Formulation	26
3.3	Residual Formulation for the PINN Model	26

3.4	Loss Function Formulation	27
3.4.1	Data Loss	27
3.4.2	Physics Loss	27
3.5	Unknown Parameter Estimation	28
3.6	Optimization Formulation	29
3.7	Role of Mathematical Formulation in This Study	29
4	Data Description and Problem Setup	30
4.1	Input Data Source	30
4.2	Synthetic / Simulated Data	30
4.3	Variables Used	31
4.3.1	Time-Series Variables (Dynamic Response)	31
4.3.2	Measured vs True Signals	31
4.3.3	Structural Parameters (Target Variables)	32
4.3.4	Modal Properties	32
4.3.5	Run Identification	33
4.4	Training and Testing Setup	33
4.5	Assumptions	33
4.6	Boundary and Initial Conditions	33
4.7	Problem Formulation	34
4.8	Sample Simulation Run (RunID = 1)	35
4.8.1	Structural Parameters for RunID = 1	35
4.8.2	Time-Series Data Illustration	36
4.8.3	Interpretation of Sample Data	36
4.9	Graphical Interpretation of Time-Series Data	37
4.9.1	Displacement Response	37
4.9.2	Velocity and Acceleration	38
4.9.3	Force Input	38
4.9.4	Effect of Measurement Noise	39
4.9.5	Engineering Insight from Plots	39
4.10	Comparison Between Data-Driven Models and PINNs	39
4.10.1	Data-Driven Approach	40
4.10.2	Physics-Informed Neural Networks (PINNs)	40
4.10.3	Relevance to This Dataset	41
4.10.4	Key Insight	41
5	Methodology and Implementation	42
5.1	Common Input Data for Stage 1 and Stage 2	42
5.2	Stage 1: Baseline PINN Framework	42

5.2.1	Overall Workflow	42
5.2.2	Data Generation / Data Loading	44
5.2.3	Preprocessing of Response Signals	45
5.2.4	PINN Architecture	46
5.2.5	Physics Residual Construction	47
5.2.6	Training Procedure	49
5.2.7	Parameter Identification Strategy	51
5.2.8	Evaluation and Visualization	52
5.3	Stage 2: Fully Blind PINN Framework	54
5.3.1	Overall Workflow	54
5.3.2	Data Generation / Data Loading	56
5.3.3	Preprocessing of Response Signals	57
5.3.4	PINN Architecture	58
5.3.5	Physics Residual Construction	60
5.3.6	Training Procedure	62
5.3.7	Parameter Identification Strategy	64
5.3.8	Evaluation and Visualization	65
5.4	Summary of Stage 2	67
6	Results and Discussion	68
6.1	Overview of Results Stage 1: Baseline PINN Framework	68
6.2	Predicted vs True Responses	68
6.2.1	Signal Reconstruction (R^2) for all Stage 1 runs	69
6.2.2	Signal Reconstruction NRMSE for all Stage 1 runs	69
6.2.3	Discussion	69
6.2.4	Estimated Structural Parameters	69
6.2.5	Detailed Stage 1 parameter-identification results for all runs	70
6.2.6	Compact parameter percentage errors for all Stage 1 runs	71
6.2.7	Discussion	71
6.3	Error Analysis	72
6.3.1	Parameter accuracy threshold counts for all Stage 1 runs	73
6.3.2	Physics residual RMS for all Stage 1 runs	73
6.3.3	Best and worst parameter errors across Stage 1 runs	74
6.3.4	Discussion	74
6.4	Convergence Behavior	75
6.4.1	Stage 1 training setup	75
6.4.2	Convergence-related summary from final evaluation	75
6.5	Discussion of Performance	76
6.5.1	Strong and weak aspects of Stage 1 performance	76

6.5.2	Discussion	76
6.6	Strengths and Limitations of the Proposed Method	77
6.6.1	Strengths observed in Stage 1	77
6.6.2	Limitations observed in Stage 1	77
6.6.3	Final Stage 1 interpretation	77
7	Results and Discussion	78
7.1	Overview of Results Stage 2: Fully Blind PINN Framework	78
7.1.1	Predicted vs True Responses	78
7.1.2	Signal reconstruction (R^2) for all Stage 2 run	79
7.1.3	Signal reconstruction NRMSE for all Stage 2 runs	79
7.1.4	Discussion	79
7.2	Estimated Structural Parameters	80
7.2.1	Parameter-level summary across all Stage 2 runs	81
7.2.2	Parameter-level summary across all Stage 2 runs	82
7.2.3	Per-run Stage 2 parameter-identification errors	83
7.2.4	Compact interpretation of Stage 2 parameter difficulty	83
7.2.5	Discussion	83
7.3	Error Analysis	84
7.3.1	Parameter accuracy threshold counts for all Stage 2 runs	84
7.3.2	Physics residual RMS for Stage 2	85
7.3.3	Best and worst Stage 2 cases	85
7.3.4	Discussion	85
7.4	Convergence Behavior	85
7.4.1	Loss and optimization settings for Stage 2	86
7.4.2	Convergence-related numerical summary for Stage 2	87
7.4.3	Discussion	87
7.5	Discussion of Performance	87
7.5.1	Overall Stage 2 performance summary	88
7.5.2	Strong and weak aspects of Stage 2 performance	89
7.5.3	Discussion	89
7.6	Strengths and Limitations of the Proposed Method	89
7.6.1	Strengths observed in Stage 2	89
7.6.2	Limitations observed in Stage 2	90
7.6.3	Final Stage 2 interpretation	90
8	Conclusion and Future Work	91
8.1	Summary of Major Findings	91
8.2	Overall Performance of the Proposed PINN Framework	92

8.3	Limitations of the Present Study	93
8.4	Comparison Between Data-Driven Models and PINNs	94
8.5	Suggested Improvements	96
8.6	Future Research Directions	97

List of Figures

2.1	Single Degree of Freedom (SDOF) system.	14
2.2	Multi Degree of Freedom (MDOF) systems.	16
2.3	Natural Frequencies	19
2.4	Mode Shapes	19
2.5	Damping Ratios	20
2.6	Physics-Informed Neural Networks (PINNs) Structures	22
3.1	Unknown Parameter Estimation	28
4.1	Measured vs True Signals	32
4.2	Problem Formulation	34
4.3	Time-Series Data Illustration	36
4.4	Displacement Response	37
4.5	Velocity and Acceleration	38
4.6	Force Input	38
4.7	Effect of Measurement Noise	39
5.1	Baseline PINN Framework Overall Workflow	43
5.2	PINN Architecture Stage 1	47
5.3	Physics Residual Construction Stage 1	48
5.4	Training Procedure Stage 1	50
5.5	Parameter Identification Strategy Stage 1	51
5.6	Fully Blind PINN Framework Overall Workflow Stage 2	55
5.7	PINN Architecture Stage 2	59
5.8	Physics Residual Construction Stage 2	61
5.9	Training procedure of Stage 2	63
5.10	Stage 2 parameter identification strategy	65
6.1	NRMSE per Signal – All Runs Stage 1	68
6.2	Signal Reconstruction (R^2) Stage 1	69
6.3	Signal Reconstruction NRMSE for all Stage 1	69
6.4	Parameter % Error All Stage 1 runs	69

6.5	Stage 1 parameter-identification results for all runs	70
6.6	parameter percentage errors for all Stage 1 runs	71
6.7	Modal property errors for all Stage 1 runs	71
6.8	Parameters Within Error Threshold-All Runs	72
6.9	Physics Residual (RMS)-All Runs	72
6.10	Parameter accuracy threshold counts for all Stage 1 runs	73
6.11	Physics residual RMS for all Stage 1 runs	73
6.12	Best and worst parameter errors across Stage 1 runs	74
6.13	Convergence Behavior	75
6.14	Convergence-related summary from final evaluation stage1	75
6.15	overall performance summary stage1	76
6.16	Strengths observed in stage1	77
7.1	NRMSE per All Runs Stage2	78
7.2	Signal Reconstruction (R^2) for all Stage 2 run	79
7.3	Signal reconstruction NRMSE for all Stage 2 runs	79
7.4	Parameter % Error All Stage 2 runs	80
7.5	Mean Prediction Error per Parameter Stage 2	80
7.6	Per- Run Summary Stage 2	81
7.7	parameter-identification results for Mass Stage 2	81
7.8	parameter-identification results for Stiffness Stage 2	81
7.9	parameter-identification results for Damping Stage 2	82
7.10	parameter-identification results for Natural Frequencies & Damping Ratios Stage 2	82
7.11	Parameter-level summary across all Stage 2	82
7.12	Per- Run Parameter Error & Accuracy Stage 2	83
7.13	Per- Run Parameter Difficulty Classification Stage 2	83
7.14	Parameters Within Error Threshold-All Runs Stage 2	84
7.15	Physics Residual (RMS)-All Runs Stage 2	84
7.16	Mean Prediction Error per Parameter Stage 2	87
7.17	Per –Run Summary Statistics Stage 2	88
7.18	Parameter Prediction Error % Stage 2	88

Chapter 1

Introduction

1.1 Structural Health Monitoring (SHM)

Structural Health Monitoring (SHM) is an interdisciplinary field of engineering that focuses on the continuous observation, assessment, and evaluation of the condition of engineering structures. The primary objective of SHM is to detect structural damage, identify changes in structural behavior, and estimate important physical parameters that govern the dynamic response of a structure.

Modern engineering structures such as bridges, buildings, aircraft components, mechanical frames, offshore platforms, and wind turbines are subjected to various environmental and operational loads throughout their service life. These loads include wind forces, traffic loads, seismic excitation, operational vibrations, and thermal effects. Over time, these loads may lead to material degradation, fatigue damage, corrosion, or structural cracking, which can significantly affect the structural integrity of the system.

Traditional structural assessment methods often rely on periodic inspections, visual examination, or localized non-destructive testing. While these methods are useful, they are limited in their ability to detect internal damage or monitor structural behavior continuously. Structural Health Monitoring provides a more advanced approach by integrating sensor technology, signal processing, and analytical modeling to evaluate structural conditions in real time.

In an SHM system, sensors such as accelerometers, displacement sensors, strain gauges, and vibration sensors are installed on the structure to measure dynamic responses. These measurements typically include displacement, velocity, acceleration, and strain. The recorded data reflects the structural behavior under operational conditions and contains valuable information about the physical properties of the structure.

The dynamic behavior of a structure is largely determined by its fundamental parameters, namely mass, stiffness, and damping. When structural damage occurs, these parameters may change. For example, the formation of cracks or deterioration of materi-

als can reduce structural stiffness, which in turn alters the vibration characteristics of the system. By analyzing changes in dynamic response data, it becomes possible to detect structural damage and assess the structural condition.

Therefore, Structural Health Monitoring combines concepts from structural dynamics, system identification, data analysis, and modern computational methods to provide reliable tools for structural safety evaluation and maintenance planning.

1.2 Importance of Structural Health Monitoring

Structural Health Monitoring plays a critical role in ensuring the safety, reliability, and longevity of engineering structures. Many infrastructures, including bridges, buildings, dams, transportation systems, and industrial equipment, are expected to operate safely for long periods under varying environmental and operational conditions.

Over time, structures may experience deterioration due to several factors such as material fatigue, corrosion, excessive loading, environmental exposure, or accidental damage. If such damage remains undetected, it may gradually propagate and eventually lead to structural failure. Structural failure can result in severe economic losses, operational disruptions, and potential threats to human safety.

Conventional structural inspection techniques often rely on manual inspections conducted at scheduled intervals. Although these inspections are valuable, they may not always detect hidden damage or identify structural degradation in its early stages. Additionally, manual inspections can be time-consuming, costly, and sometimes impractical for large-scale structures.

Structural Health Monitoring addresses these limitations by enabling continuous monitoring of structural behavior through automated sensing systems. By collecting and analyzing real-time data from sensors installed on the structure, SHM systems can identify abnormal structural responses and detect potential damage at an early stage.

One of the key advantages of SHM is its ability to detect changes in the dynamic characteristics of a structure. The vibration behavior of a structure is governed by its physical properties such as mass, stiffness, and damping. When damage occurs, these parameters change, which may cause variations in the natural frequencies, mode shapes, or damping characteristics of the system. Monitoring these changes provides a reliable way to detect structural damage.

In addition to damage detection, SHM also supports system identification and parameter estimation, allowing engineers to quantify structural properties and evaluate the overall health of the system. These capabilities are essential for developing effective maintenance strategies and ensuring long-term structural safety.

1.3 Why Multi-Degree-of-Freedom (MDOF) Systems Are Used

In structural dynamics, simplified mathematical models are often used to represent the behavior of complex structures. The simplest dynamic model is the Single Degree of Freedom (SDOF) system, where the motion of the entire structure is described by a single displacement coordinate.

Although SDOF models are useful for basic vibration analysis, they are often insufficient to accurately represent real engineering structures. Most structures consist of multiple interconnected components that can move independently under dynamic loading conditions. As a result, their behavior cannot be described using only one displacement coordinate.

To represent the dynamic behavior of such systems more accurately, Multi-Degree-of-Freedom (MDOF) models are used. In an MDOF system, multiple displacement coordinates are required to describe the motion of different parts of the structure. Each coordinate represents the displacement of a particular mass or structural component.

For an n -degree-of-freedom system, the displacement vector can be expressed as

$$u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_n(t) \end{bmatrix}$$

where $u_i(t)$ represents the displacement of the i^{th} degree of freedom.

MDOF models provide several advantages for structural analysis. They allow engineers to capture the interaction between different structural components and accurately represent complex vibration patterns. These models are particularly important in Structural Health Monitoring because damage occurring in one part of the structure may influence the dynamic behavior of other components.

By analyzing the vibration response of an MDOF system, it is possible to extract important information about the structural parameters and detect potential damage within the system.

1.4 Why Physics-Informed Neural Networks (PINNs) Are Useful

In recent years, machine learning techniques have been increasingly applied to structural health monitoring and system identification problems. Neural networks are particularly

useful for modeling complex nonlinear relationships between input data and structural responses.

However, traditional data-driven neural networks rely entirely on training data to learn system behavior. In many engineering applications, obtaining large amounts of high-quality data may be difficult or expensive. Furthermore, purely data-driven models may produce predictions that violate the physical laws governing the system.

Physics-Informed Neural Networks (PINNs) provide a powerful solution to these challenges by incorporating physical laws directly into the neural network training process. In a PINN framework, the governing equations of the physical system are embedded into the loss function used during training. This ensures that the predicted solutions satisfy both the observed data and the underlying physical principles.

Instead of learning solely from data, PINNs combine two sources of information:

1. observational data from experiments or simulations
2. governing physical equations describing the system behavior

By enforcing physical constraints during training, PINNs can produce physically consistent predictions even when the available data is limited.

In structural dynamics problems, the governing equations of motion describe the relationship between displacement, velocity, acceleration, and structural parameters. By incorporating these equations into the neural network training process, PINNs can be used to estimate unknown parameters such as mass, stiffness, and damping.

This capability makes PINNs particularly suitable for solving inverse problems in structural dynamics, where the goal is to estimate system parameters from measured response data.

1.5 Research Objective

The primary objective of this project is to develop a Physics-Informed Neural Network framework for the modeling and parameter identification of a Multi-Degree-of-Freedom structural system.

The specific objectives of the study include:

1. Formulating the governing equations of motion for a multi-degree-of-freedom structural system.
2. Developing a neural network model capable of representing the structural dynamic response.
3. Incorporating physical constraints derived from structural dynamics equations into the neural network training process.

4. Estimating unknown structural parameters such as mass, stiffness, and damping using response data.
5. Evaluating the accuracy and effectiveness of the proposed approach by comparing predicted results with reference values.

Through this approach, the project aims to demonstrate the potential of Physics-Informed Neural Networks as a powerful tool for Structural Health Monitoring and system identification.

1.6 Scope of the Work

This work focuses on the application of Physics-Informed Neural Networks to modeling and parameter identification of a Multi-Degree-of-Freedom structural system using dynamic response data.

The study is based on a computational framework in which structural response data is used together with the governing equations of motion to train a neural network model. The developed model aims to estimate structural parameters while ensuring that the predicted responses satisfy the physical laws governing structural dynamics.

The report covers the theoretical background of structural dynamics and Structural Health Monitoring, the mathematical formulation of the governing equations, the development of the PINN-based modeling approach, the implementation methodology, and the analysis of obtained results.

The work primarily focuses on simulated or available response data and computational modeling techniques. Experimental validation and real-world deployment of SHM systems are beyond the scope of this study but may be considered potential directions for future research.

Chapter 2

Structural Health Monitoring and Structural Dynamics Fundamentals

2.1 Structural Dynamics

Structural dynamics is the branch of mechanics that studies how structures respond to dynamic loads over time.

Unlike static analysis, where loads remain constant, dynamic analysis considers time-dependent forces.

The response of a dynamic system depends on three fundamental physical properties:

Property	Symbol	Physical Meaning
Mass	m	resistance to acceleration
Damping	c	Energy dissipation mechanism
Stiffness	k	Resistance to deformation

These three parameters determine the vibration behavior of a structure.

2.2 Single Degree of Freedom (SDOF) System

2.2.1 Basic Components of an SDOF System

The simplest structural dynamic system is the Single Degree of Freedom (SDOF) system.

A typical SDOF system consists of:

- one mass
- one spring
- one damper

Example model:



Figure 2.1: Single Degree of Freedom (SDOF) system.

2.2.2 Equation of Motion of an SDOF System

Using Newton's Second Law of Motion:

$$F = ma$$

Where

Symbol	Meaning
F	Applied force
m	Mass
a	Acceleration

Acceleration is the second derivative of displacement:

$$a = \ddot{u}(t)$$

Velocity:

$$v = \dot{u}(t)$$

Considering all forces acting on the mass:

1. Inertia force $\rightarrow m\ddot{u}$
2. Damping force $\rightarrow c\dot{u}$
3. Spring force $\rightarrow ku$

Applying equilibrium:

$$m\ddot{u}(t) + c\dot{u}(t) + ku(t) = f(t)$$

where

Term	Meaning
$m\ddot{u}$	inertia force
$c\dot{u}$	damping force
ku	restoring force
$f(t)$	external force

2.2.3 Natural Frequency

If the system vibrates without external force, then:

$$f(t) = 0$$

The equation becomes:

$$m\ddot{u} + ku = 0$$

Assuming harmonic motion:

$$u(t) = A \sin(\omega t)$$

Substituting into the equation gives:

$$\omega = \sqrt{\frac{k}{m}}$$

where ω = natural circular frequency.

The natural frequency in Hertz is:

$$f = \frac{\omega}{2\pi}$$

2.2.4 Damping Ratio

Damping describes how vibration energy is dissipated.

The damping ratio is defined as:

$$\zeta = \frac{c}{2\sqrt{km}}$$

Where

Symbol	Meaning
c	damping coefficient
k	stiffness
m	mass

The damping ratio determines the vibration behavior:

Damping Ratio	System Behavior
$\zeta = 0$	undamped vibration
$0 < \zeta < 1$	underdamped
$\zeta = 1$	critically damped
$\zeta > 1$	overdamped

2.3 Multi Degree of Freedom (MDOF) Systems

2.3.1 Concept of MDOF Systems

Real structures cannot be accurately represented by a single mass. Instead, they consist of multiple masses interacting with each other.

Such systems are called Multi Degree of Freedom (MDOF) systems.

Example: Two Degree of Freedom system

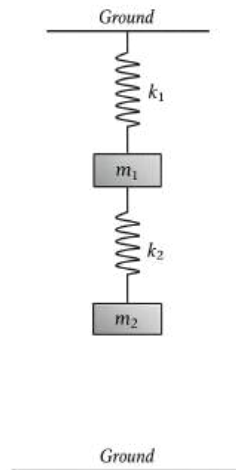


Figure 2.2: Multi Degree of Freedom (MDOF) systems.

Each mass can move independently.

Therefore, the system has multiple displacement coordinates:

$$u_1(t), u_2(t), \dots, u_n(t)$$

These are called degrees of freedom.

2.3.2 Equation of Motion of MDOF System

For an MDOF system, the equations can be written in matrix form:

$$M u''(t) + C u'(t) + K u(t) = f(t)$$

Where

Matrix	Description
M	mass matrix
C	Damping matrix
K	Stiffness matrix
$u(t)$	Displacement vector
$f(t)$	External force vector

Example displacement vector:

$$u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \end{bmatrix}$$

Mass matrix:

$$M = \begin{bmatrix} m_1 & 0 \\ 0 & m_2 \end{bmatrix}$$

Stiffness matrix:

$$K = \begin{bmatrix} k_1 + k_2 & -k_2 \\ -k_2 & k_2 \end{bmatrix}$$

This system represents coupled differential equations describing structural motion.

2.3.3 Modal Properties

The dynamic behavior of multi-degree-of-freedom (MDOF) systems can be effectively analyzed using modal analysis, which provides insight into the natural vibration characteristics of a structure. Modal analysis decomposes the complex motion of an MDOF system into a set of independent vibration modes.

For an undamped free vibration system without external forces, the equation of motion can be written as

$$M\ddot{u}(t) + Ku(t) = 0$$

where

- M represents the mass matrix,
- K represents the stiffness matrix,
- $u(t)$ represents the displacement vector.

To determine the natural vibration characteristics, a harmonic solution is assumed in the form

$$u(t) = \phi e^{i\omega t}$$

where

- ϕ is the mode shape vector and
- ω is the circular natural frequency.

Substituting this assumed solution into the governing equation yields

$$(K - \omega^2 M)\phi = 0$$

This expression represents a generalized eigenvalue problem. For a non-trivial solution to exist, the determinant of the matrix must be zero:

$$\det(K - \omega^2 M) = 0$$

Solving this equation provides a set of eigenvalues and eigenvectors.

The eigenvalues correspond to the natural frequencies of the system, while the eigenvectors represent the mode shapes.

Natural Frequencies

Natural frequencies are the frequencies at which a system tends to vibrate when it is disturbed from its equilibrium position. These frequencies depend on structural properties such as mass and stiffness.

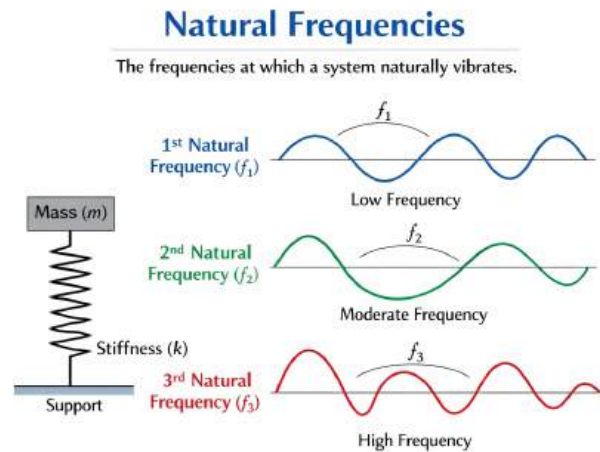


Figure 2.3: Natural Frequencies

Mode Shapes

Mode shapes describe the deformation pattern of the structure during vibration at a particular natural frequency. Each mode shape represents a distinct vibration pattern of the structure.

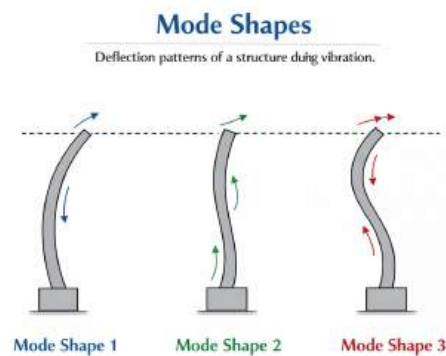


Figure 2.4: Mode Shapes

Damping Ratios

In practical systems, damping is present and causes the vibration amplitude to decrease over time. The damping ratio describes the rate at which the vibration energy dissipates in the system.

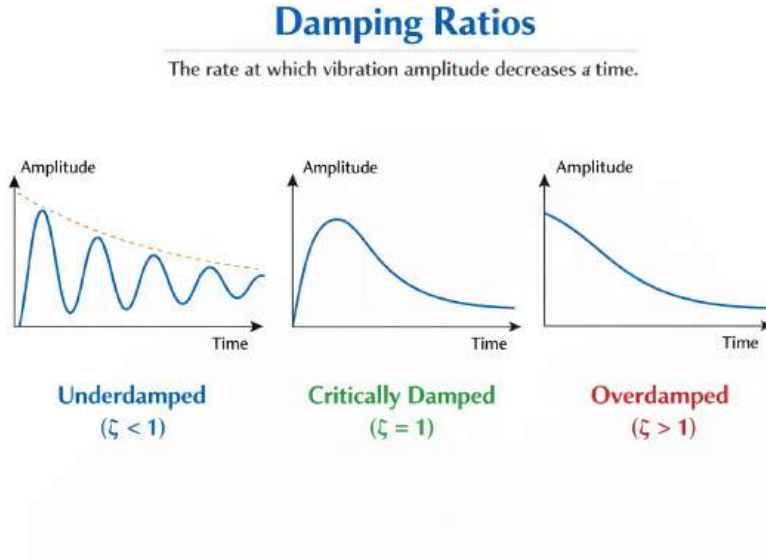


Figure 2.5: Damping Ratios

Changes in natural frequencies or mode shapes may indicate structural damage, which makes modal properties extremely important for Structural Health Monitoring applications.

2.4 Structural Health Monitoring Concepts

Structural Health Monitoring (SHM) is a methodology used to evaluate the condition of structures by analyzing their dynamic responses. SHM systems rely on measured data obtained from sensors installed on the structure to detect damage and assess structural performance.

One of the main objectives of SHM is damage detection, which involves identifying the presence and location of structural damage. Structural damage may occur due to material fatigue, environmental effects, excessive loading, or aging of structural components.

Another important task in SHM is system identification, which refers to the process of determining the physical parameters of a structure using observed response data. These parameters typically include mass, stiffness, and damping coefficients.

Closely related to system identification is parameter estimation, where the goal is to estimate unknown structural parameters that govern the dynamic behavior of the system. Changes in these parameters often indicate the presence of structural damage or degradation.

The effectiveness of SHM systems depends largely on the availability of reliable sensor measurements. Sensors commonly used in structural monitoring include accelerometers, displacement sensors, and strain gauges. These sensors measure quantities such as displacement, velocity, acceleration, and strain.

Among these measurements, displacement and acceleration data are particularly useful because they directly reflect the dynamic response of the structure. By analyzing these signals, it becomes possible to infer the internal state of the structural system and identify potential damage.

Modern SHM approaches increasingly combine traditional structural dynamics with advanced computational techniques such as machine learning and data-driven modeling to improve the accuracy of damage detection and parameter estimation.

2.5 Physics-Informed Neural Networks (PINNs)

Physics-Informed Neural Networks (PINNs) are a class of machine learning models that integrate physical laws into the training process of neural networks. Unlike traditional neural networks that rely purely on data, PINNs enforce the governing equations of the physical system as part of the learning process.

In the context of structural dynamics, the neural network is used to approximate the displacement response of the structure as a function of time. The predicted displacement can be represented as

$$u(t) \approx \hat{u}(t)$$

Where $\hat{u}(t)$ denotes the neural network prediction.

To ensure that the predicted solution satisfies the physical laws governing the structural system, a physics residual is defined using the equation of motion:

$$R(t) = M u''(t) + C u'(t) + K u(t) - f(t)$$

Ideally, the residual should be zero if the predicted displacement perfectly satisfies the governing equation.

During training, the neural network minimizes a loss function composed of two main components.

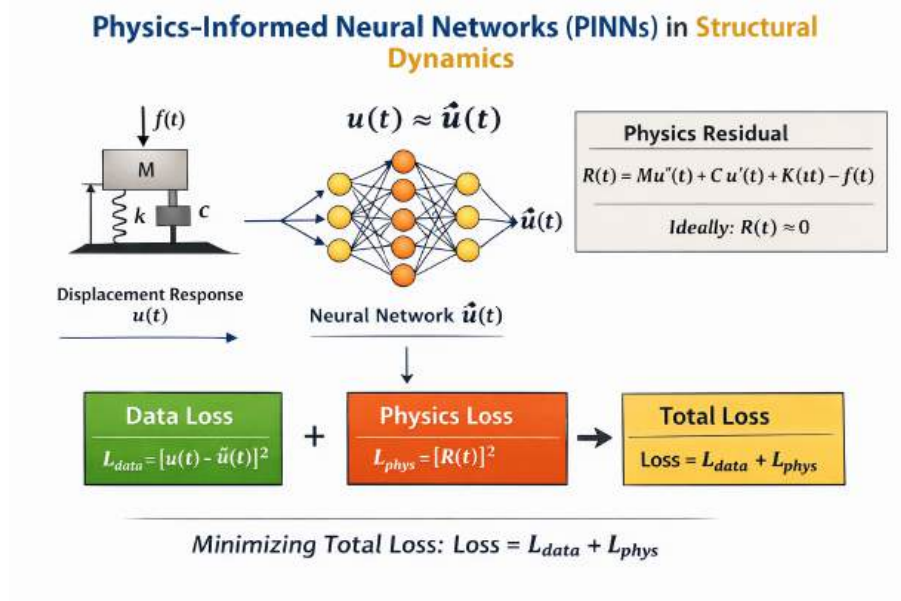


Figure 2.6: Physics-Informed Neural Networks (PINNs) Structures

2.5.1 Data Loss

The data loss measures the difference between the predicted displacement and the observed displacement data:

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^N (u_{\text{pred}}(t_i) - u_{\text{true}}(t_i))^2$$

2.5.2 Physics Loss

The physics loss enforces the governing differential equation of the structural system:

$$L_{\text{physics}} = \frac{1}{N} \sum_{i=1}^N R(t_i)^2$$

2.5.3 Total Loss Function

The total loss used for training the PINN model is defined as

$$L = L_{\text{data}} + \lambda L_{\text{physics}}$$

Where λ is a weighting parameter that balances the contribution of the physics constraint and the data fitting.

By incorporating physical laws into the learning process, PINNs ensure that the predicted solutions remain physically consistent even when the available data is limited. This

capability makes PINNs particularly suitable for solving inverse problems in structural dynamics and structural health monitoring.

2.6 Importance of MDOF Models in Structural Health Monitoring

Multi-degree-of-freedom models play an important role in Structural Health Monitoring because they provide a realistic representation of complex structural systems. Real-world structures consist of multiple interconnected components that interact with each other during vibration.

MDOF models allow engineers to capture these interactions by representing the structure as a system of multiple masses, springs, and dampers. Each degree of freedom represents the motion of a specific structural component.

In SHM applications, structural damage often causes localized changes in stiffness or damping properties. These changes affect the global dynamic behavior of the structure, including natural frequencies, mode shapes, and vibration amplitudes.

By analyzing the dynamic response of an MDOF system, it becomes possible to estimate unknown structural parameters and detect changes in structural properties. This approach forms the basis of many SHM techniques used for damage detection and system identification.

Therefore, MDOF models provide a powerful framework for analyzing structural vibrations and identifying potential structural damage using measured response data.

2.7 Connection to the Present Project

In this project, the structural system is modeled as a multi-degree-of-freedom dynamic system governed by the equation of motion

$$M u''(t) + C u'(t) + K u(t) = f(t)$$

where M , C , and K represent the mass, damping, and stiffness matrices of the system.

The objective of the project is not only to simulate the structural response but also to identify unknown structural parameters from available response data. This problem is known as an inverse structural dynamics problem, where the goal is to determine system parameters from observed structural responses.

To solve this problem, a Physics-Informed Neural Network framework is employed. The neural network is trained using both observed response data and the governing equations of motion. By enforcing physical constraints during training, the model ensures

that the predicted structural response satisfies the laws of structural dynamics.

Through this approach, the project aims to develop a computational framework capable of estimating structural parameters such as stiffness and damping while accurately reproducing the dynamic behavior of the structure.

This combination of physics-based modeling and machine learning provides an effective tool for structural system identification and structural health monitoring applications.

Chapter 3

Mathematical Formulation

This chapter presents the mathematical formulation of the structural dynamic system and the Physics-Informed Neural Network (PINN) framework used in this study. The governing equations of motion for the multi-degree-of-freedom (MDOF) system are first introduced. These equations describe the relationship between structural displacement, velocity, acceleration, and external forces. The formulation is then extended to construct the physics-based residual used in the PINN model. Finally, the loss function and optimization framework used to estimate the unknown structural parameters are described.

3.1 Governing Equations of Structural Dynamics

The dynamic behavior of a multi-degree-of-freedom structural system can be described using the equation of motion derived from Newton's second law.

For an MDOF system, the governing equation can be expressed in matrix form as

$$M u''(t) + C u'(t) + K u(t) = f(t)$$

Where

Symbol	Description
M	Mass matrix
C	Damping matrix
K	Stiffness matrix
$u(t)$	Displacement vector
$u'(t)$	Velocity vector
$u''(t)$	Acceleration vector
$f(t)$	External force vector

The displacement vector for an n -degree-of-freedom system is

$$u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_n(t) \end{bmatrix}$$

Each component represents the displacement of an individual structural mass.

3.2 Direct Displacement Formulation

In many structural dynamic problems, the displacement response $u(t)$ is treated as the primary variable describing the system motion. The velocity and acceleration can be obtained by differentiating the displacement with respect to time.

Velocity:

$$\dot{u}(t) = \frac{du(t)}{dt}$$

Acceleration:

$$\ddot{u}(t) = \frac{d^2u(t)}{dt^2}$$

Substituting these expressions into the governing equation gives

$$M \frac{d^2u}{dt^2} + C \frac{du}{dt} + Ku = f(t)$$

This differential equation governs the dynamic response of the structural system.

3.3 Residual Formulation for the PINN Model

In the Physics-Informed Neural Network framework, the neural network is used to approximate the displacement response of the structure.

$$u(t) \approx \hat{u}(t)$$

Where

$\hat{u}(t)$ represents the predicted displacement produced by the neural network.

To enforce the governing equation of motion, a physics residual is defined. The residual measures the deviation of the predicted solution from the governing equation.

$$R(t) = M u''(t) + C u'(t) + K u(t) - f(t)$$

If the predicted displacement satisfies the governing equation perfectly, the residual becomes zero.

$$R(t) = 0$$

Therefore, the PINN training process attempts to minimize the residual.

3.4 Loss Function Formulation

The neural network training process is based on minimizing a loss function that combines both data consistency and physical constraints.

The total loss function is defined as

$$L = L_{\text{data}} + \lambda L_{\text{physics}}$$

Where

Term	Meaning
L_{data}	Data loss
L_{physics}	Physics loss
λ	weighting parameter

3.4.1 Data Loss

The data loss measures the difference between predicted and observed structural responses.

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^N (u_{\text{pred}}(t_i) - u_{\text{true}}(t_i))^2$$

This term ensures that the neural network prediction fits the observed data.

3.4.2 Physics Loss

The physics loss ensures that the predicted solution satisfies the governing differential equation.

$$L_{\text{physics}} = \frac{1}{N} \sum_{i=1}^N R(t_i)^2$$

where $R(t_i)$ is the residual of the governing equation evaluated at the time point t_i .

3.5 Unknown Parameter Estimation

In this project, certain structural parameters are treated as unknown variables that need to be estimated from the observed response data.

Typical parameters include

- stiffness coefficients
- damping coefficients
- mass parameters

Instead of assigning fixed values to these parameters, they are treated as trainable variables in the neural network model.

Let the unknown parameters be represented as

$$\theta = [k_1, k_2, c_1, c_2, \dots]$$

During training, the neural network simultaneously learns

1. the structural response function $u(t)$
2. the unknown parameters θ

The optimization process adjusts these parameters to minimize the total loss function.

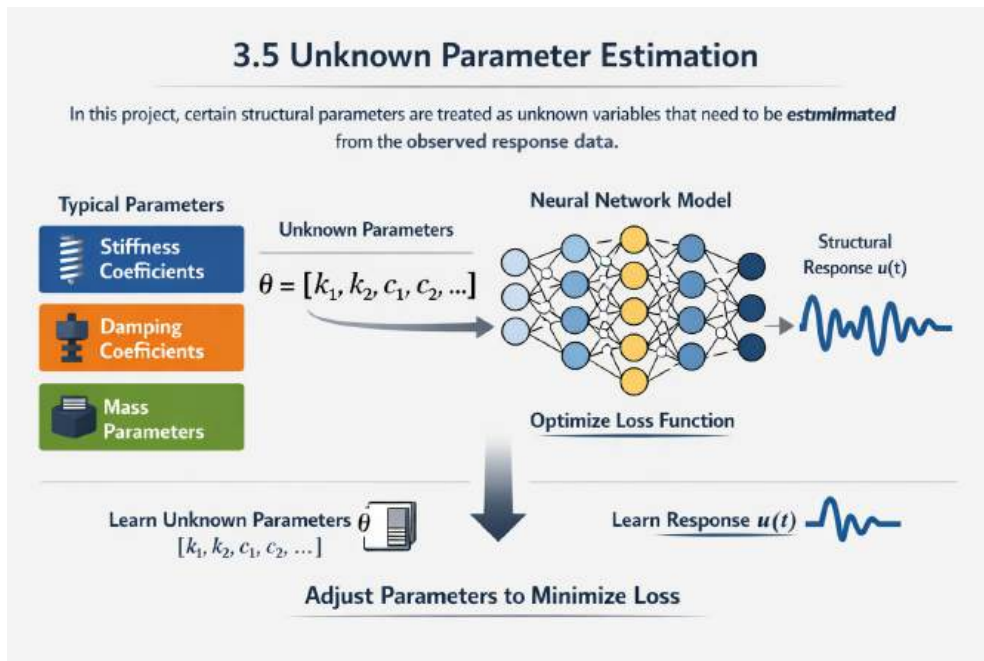


Figure 3.1: Unknown Parameter Estimation

3.6 Optimization Formulation

The training objective can therefore be expressed as the following optimization problem:

$$\min_{\theta, NN} L(\theta)$$

Where

Symbol	Description
NN	Neural network parameters
θ	Structural parameters

The optimization process aims to find the values of NN and θ that minimize the total loss function.

In practice, gradient-based optimization algorithms such as

- Adam optimizer
- stochastic gradient descent
- L-BFGS

are used to train the model.

Through this optimization process, the Physics-Informed Neural Network learns a displacement function that both fits the observed data and satisfies the governing structural dynamics equations.

3.7 Role of Mathematical Formulation in This Study

The mathematical formulation presented in this chapter provides the theoretical basis for the computational implementation described in the following chapters. By combining structural dynamic equations with neural network approximation, the proposed PINN framework enables the estimation of unknown structural parameters while maintaining physical consistency.

This approach forms the foundation for the methodology and implementation steps described in the subsequent chapters.

Chapter 4

Data Description and Problem Setup

4.1 Input Data Source

The dataset used in this study is generated through high-fidelity numerical simulations of a two-degree-of-freedom (2DOF) structural system. The system represents a coupled mass–spring–damper model subjected to external dynamic excitation. Unlike experimental datasets, the data is entirely synthetic, allowing full control over structural parameters and enabling precise ground-truth labeling.

Each simulation run corresponds to a unique configuration of structural parameters, including mass, stiffness, and damping characteristics. The simulations are performed by solving the governing equations of motion of the 2DOF system, ensuring that all generated signals strictly follow the laws of structural dynamics.

The use of synthetic data is particularly advantageous for inverse modeling tasks, as it provides complete knowledge of the true system parameters, which are otherwise difficult to obtain in real-world scenarios.

4.2 Synthetic / Simulated Data

The dataset consists of 216 independent simulation runs, each representing a distinct structural system with varying physical properties. For each run, the dynamic response of the system is computed over a fixed time interval using a uniform time discretization.

The time variable ranges from an initial state at $t = 0$ with a constant time step of $\Delta t = 0.05$ seconds. At each time step, the system response is recorded, resulting in a time-series dataset for each run.

The simulations incorporate external excitation forces applied to both degrees of freedom, denoted as $F_1(t)$ and $F_2(t)$. These forces act as inputs to the system and induce coupled dynamic responses across the two masses.

The dataset captures the complete evolution of the system response, including dis-

placement, velocity, and acceleration, making it suitable for both data-driven and physics-informed modeling approaches.

4.3 Variables Used

The dataset contains a comprehensive set of variables that describe both the dynamic response and the underlying physical properties of the system.

4.3.1 Time-Series Variables (Dynamic Response)

For each simulation run, the following time-dependent variables are recorded:

- **Displacement:** $u_1(t), u_2(t)$
- **Velocity:** $v_1(t), v_2(t)$
- **Acceleration:** $a_1(t), a_2(t)$
- **External Forces:** $F_1(t), F_2(t)$

These variables represent the observable behavior of the system. Displacement describes the positional response, velocity represents the rate of change of displacement, and acceleration reflects the system's response to internal and external forces.

4.3.2 Measured vs True Signals

In addition to the true displacement signals, the dataset includes:

- **Measured displacement:** $u_1^{\text{meas}}(t), u_2^{\text{meas}}(t)$

These signals incorporate measurement noise, simulating real-world sensor observations. This distinction between true and measured signals is crucial for structural health monitoring applications, where only noisy measurements are typically available.

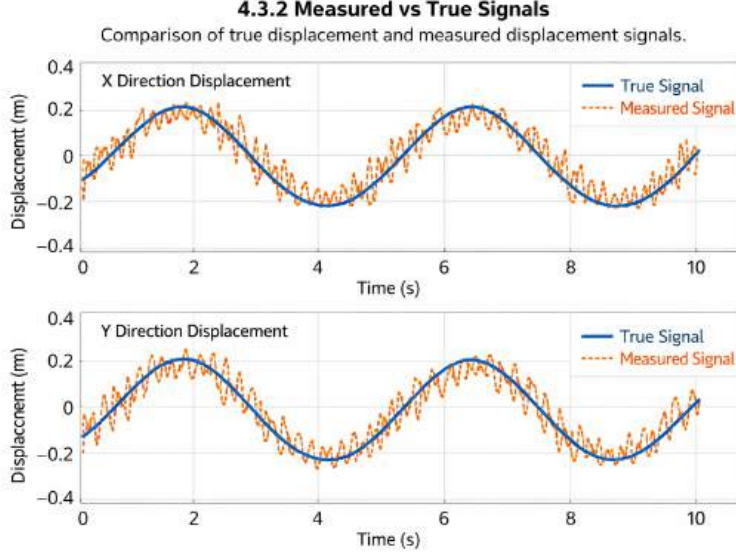


Figure 4.1: Measured vs True Signals

4.3.3 Structural Parameters (Target Variables)

Each simulation run is associated with a set of constant structural parameters:

- **Mass:** m_1, m_2
- **Stiffness:** k_1, k_2
- **Damping coefficients:** α, β (Rayleigh damping)

The damping matrix is defined as:

$$C = \alpha M + \beta K$$

These parameters fully define the physical characteristics of the structural system and remain constant throughout each simulation run.

4.3.4 Modal Properties

The dataset also includes derived modal quantities:

- **Natural frequencies:** ω_1, ω_2
- **Damping ratios:** ζ_1, ζ_2

These values are computed from the system matrices and provide insight into the vibrational characteristics of the structure.

4.3.5 Run Identification

Each data point is associated with a RunID, which uniquely identifies the simulation run. All time steps corresponding to a specific run share identical structural parameters.

4.4 Training and Testing Setup

The dataset is structured such that each simulation run represents an independent sample for model training and evaluation. The time-series data for each run forms the input, while the corresponding structural parameters serve as target outputs.

For data-driven approaches, the input features consist of sequences of dynamic signals, typically including measured displacement, velocity, acceleration, and applied forces. These sequences are used to train models to learn the mapping from observed responses to underlying structural parameters.

For Physics-Informed Neural Networks (PINNs), the full time-domain data is utilized. In addition to fitting the observed data, the model is constrained by the governing equations of motion, enabling physically consistent parameter estimation.

The dataset can be divided into training and testing subsets based on simulation runs to ensure proper evaluation of model generalization.

4.5 Assumptions

The following assumptions are adopted in the data generation and modeling process:

- The structural system is linear and time-invariant
- Mass, stiffness, and damping parameters remain constant within each simulation run
- External forces are known and accurately applied
- Measurement noise is limited to displacement observations
- The system behavior strictly follows classical structural dynamics principles

These assumptions simplify the modeling process while preserving the essential physical characteristics of the system.

4.6 Boundary and Initial Conditions

All simulations are performed under consistent initial conditions, where the system is assumed to start from rest:

- Initial displacement: $u_1(0) = u_2(0) = 0$
- Initial velocity: $v_1(0) = v_2(0) = 0$

Boundary conditions are implicitly defined by the structural configuration, typically representing fixed supports or constrained motion at the ends of the system.

These conditions ensure that the simulation results are consistent and physically meaningful across all runs.

4.7 Problem Formulation

The objective of this study is to estimate the structural parameters of the system from observed dynamic responses. This is formulated as an inverse problem.

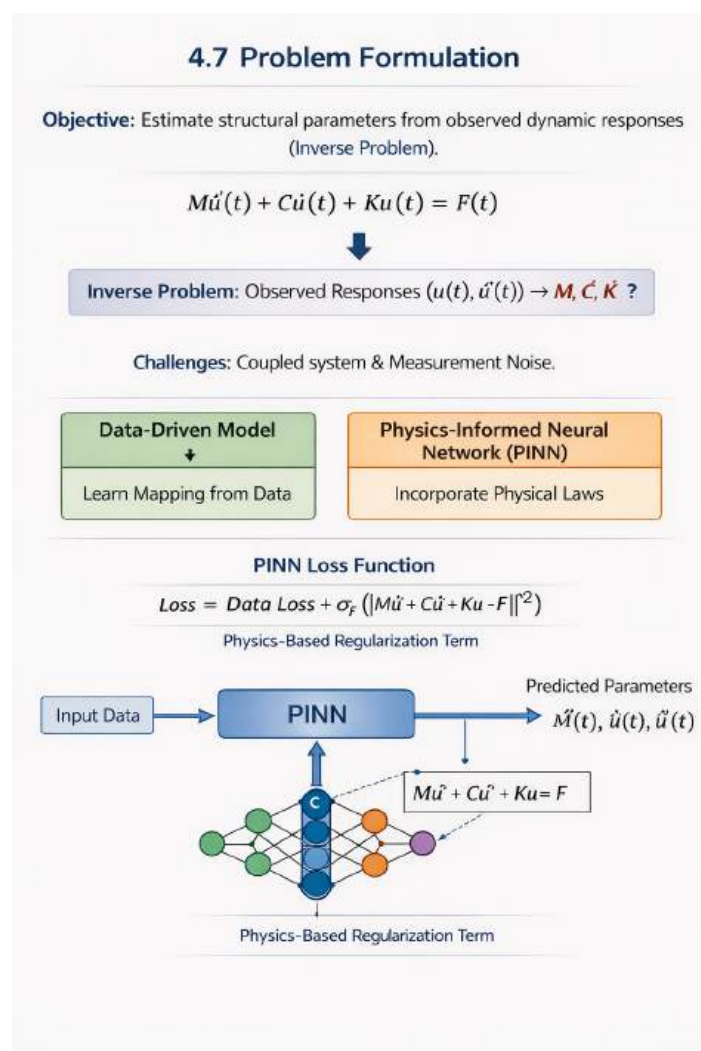


Figure 4.2: Problem Formulation

In the forward problem, the system response is computed using the equation of motion:

$$M\ddot{u}(t) + C\dot{u}(t) + Ku(t) = F(t)$$

In contrast, the inverse problem aims to determine the unknown parameters M , K , C from observed signals.

This can be expressed as the following mapping:

$$\{F(t), u^{\text{meas}}(t), \dot{u}(t), a(t)\} \rightarrow \{m_1, m_2, k_1, k_2, \alpha, \beta\}$$

The problem is inherently challenging due to the coupled nature of the system and the presence of measurement noise.

Two modeling approaches are considered:

- **Data-driven models**, which learn the mapping directly from data
- **Physics-Informed Neural Networks (PINNs)**, which incorporate physical laws into the learning process

The PINN approach enforces the governing equations through a physics-based loss function:

$$L = L_{\text{data}} + \lambda L_{\text{physics}}$$

This formulation ensures that the predicted parameters are not only consistent with observed data but also satisfy the underlying physical principles.

4.8 Sample Simulation Run (RunID = 1)

To illustrate the structure and characteristics of the dataset, a representative sample corresponding to RunID = 1 is presented. This example provides insight into the time-series behavior of the system and the associated structural parameters.

4.8.1 Structural Parameters for RunID = 1

For this simulation run, the structural parameters are constant across all time steps and are given as:

- **Mass:** $m_1 = 800$, $m_2 = 1600$
- **Stiffness:** $k_1 = 160000$, $k_2 = 400000$
- **Rayleigh damping coefficients:**

$$- \alpha = 2.904 \times 10^{-5}$$

$$- \beta = 0.3464$$

The corresponding modal properties are:

- **Natural frequencies:**

$$- \omega_1 = 10.72 \text{ rad/s}$$

$$- \omega_2 = 20.86 \text{ rad/s}$$

- **Damping ratios:**

$$- \zeta_1 = 0.0163$$

$$- \zeta_2 = 0.0086$$

These parameters define the dynamic characteristics of the system for this run.

4.8.2 Time-Series Data Illustration

A portion of the time-series data for RunID = 1 is shown in Table 4.1.

Table 4.1: Sample Time-Series Data (RunID = 1)

2-DOF Dynamic Response Data (Complete)																					
Time(s)	F1(N)	F2(N)	u1(m)	u2(m)	v1(m/s)	v1(m/s)	a2(m/s²)	a2(m/s²)	u1_meas(m)	m1(kg)	m1(kg)	k1(N/m)	k2(N/m)	alpha	beta_ray(rad/s)	omega1(rad/s)	omega2(rad/s)	zeta1	zeta2	ID	
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.346	10.72	20.86	0.0163	0.0086	1
0.01	125.00	-83.33	1.56e-06	-2.65e-04	0.156	-0.265	19.53	-33.15	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.342	20.86	0.0163	0.0086	2	
0.02	250.00	-166.67	6.24e-06	-4.24e-04	0.312	-0.529	39.05	-66.30	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.346	20.86	0.0163	0.0086	3	
0.03	375.00	-250.00	1.41e-05	-6.89e-04	0.468	-0.830	58.58	-99.45	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.346	20.86	0.0163	0.0086	4	
0.04	500.00	-333.33	2.44e-05	-1.04e-03	0.624	-1.177	78.10	-132.60	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.346	20.86	0.0163	0.0086	5	
0.05	625.00	-416.67	3.24e-05	-1.47e-03	0.624	-1.177	78.10	-132.60	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.346	20.86	0.0163	0.0086	6	
0.06	625.00	-416.67	3.75e-05	-1.47e-03	0.780	-1.569	97.63	-165.75	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.342	20.86	0.0163	0.0086	7	
0.07	750.00	-500.00	5.31e-05	-1.96e-03	0.936	-1.997	117.15	-198.90	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.346	20.86	0.0163	0.0086	8	
0.07	875.00	-583.33	7.19e-05	-2.50e-03	1.092	-2.459	136.68	-232.05	6.24e-06	-2.12e-04	800	1600	160000	400000	2.90e-05	0.342	20.86	0.0163	0.0086	9	
0.08	1000.00	-666.67	9.38e-05	-3.38e-03	1.092	-2.459	136.68	-232.05	6.24e-06	-2.12e-04	800	1600	161000	400000	2.90e-05	0.342	20.86	0.0163	0.0086	9	

Figure 4.3: Time-Series Data Illustration

4.8.3 Interpretation of Sample Data

The sample data clearly demonstrates the dynamic evolution of the system response under external excitation. At the initial time step ($t = 0$), all state variables are zero, consistent with the assumption of initial rest conditions. However, the measured displacement signals u^{meas} show small non-zero values due to simulated measurement noise.

As time progresses, the applied forces $F_1(t)$ and $F_2(t)$ excite the system, resulting in increasing displacement, velocity, and acceleration responses. The oscillatory nature of the displacement signals reflects the inherent vibrational characteristics of the system, governed by its natural frequencies.

The acceleration signals exhibit higher variability compared to displacement and velocity, indicating their sensitivity to both system parameters and external excitation. This makes acceleration particularly informative for parameter identification tasks.

A comparison between true displacement u and measured displacement u^{meas} highlights the presence of noise, which introduces realistic challenges similar to those encountered in real-world structural health monitoring systems.

4.9 Graphical Interpretation of Time-Series Data

To further understand the dynamic behavior of the system, the time-series responses of displacement, velocity, acceleration, and applied forces can be visualized as functions of time. These plots provide intuitive insight into the physical characteristics of the system.

4.9.1 Displacement Response

The displacement signals $u_1(t)$ and $u_2(t)$ exhibit oscillatory behavior, characteristic of vibratory systems. The frequency of oscillation corresponds to the natural frequencies of the system, while the amplitude is influenced by both the excitation force and damping effects.

In the sample run, the displacement starts from zero and gradually evolves as the external force excites the system. The presence of two distinct modes can lead to coupled oscillations between the two masses.

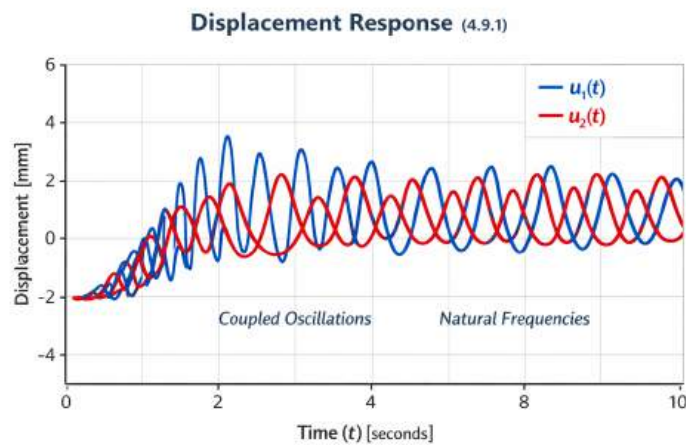


Figure 4.4: Displacement Response

4.9.2 Velocity and Acceleration

Velocity $v(t)$ represents the rate of change of displacement and follows a similar oscillatory pattern with phase differences. Acceleration $a(t)$, being the second derivative of displacement, shows sharper variations and is highly sensitive to system dynamics.

Acceleration signals are particularly important because they directly relate to the governing equation of motion and contain rich information about system parameters.

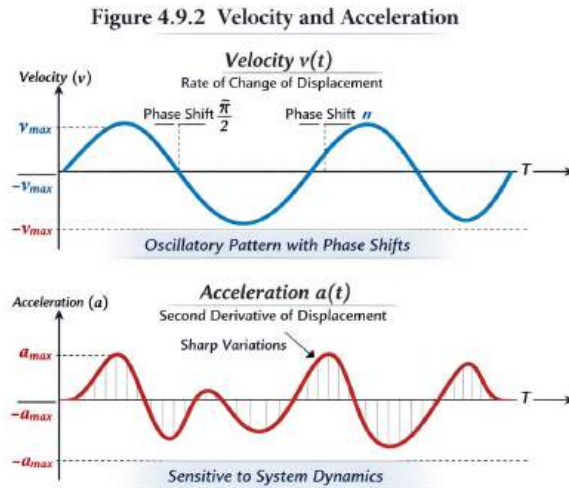


Figure 4.5: Velocity and Acceleration

4.9.3 Force Input

The external forces $F_1(t)$ and $F_2(t)$ act as driving inputs to the system. The variation in force over time determines how energy is introduced into the system, influencing both amplitude and frequency content of the response.

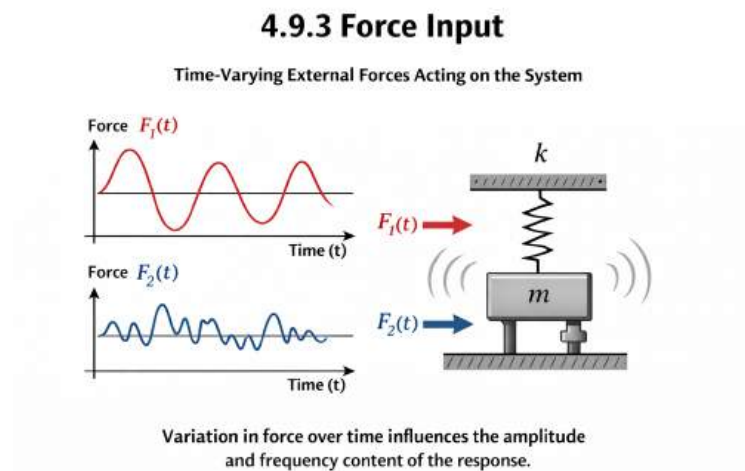


Figure 4.6: Force Input

4.9.4 Effect of Measurement Noise

A comparison between true displacement $u(t)$ and measured displacement $u^{\text{meas}}(t)$ reveals small deviations caused by noise. Although the noise magnitude is relatively small, it introduces realistic uncertainty into the dataset, making the parameter identification problem more challenging.

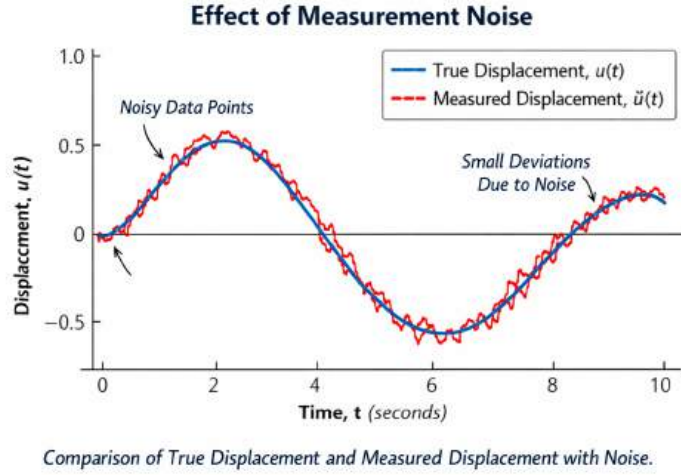


Figure 4.7: Effect of Measurement Noise

4.9.5 Engineering Insight from Plots

From these plots, several important observations can be made:

- The system exhibits multi-modal oscillatory behavior
- Damping causes gradual reduction in amplitude
- Higher stiffness results in faster oscillations
- Acceleration captures rapid system changes and is highly informative

These observations validate that the dataset accurately represents realistic structural dynamics.

4.10 Comparison Between Data-Driven Models and PINNs

In this study, two different modeling approaches are considered for solving the inverse problem of structural parameter identification: data-driven models and Physics-Informed Neural Networks (PINNs).

4.10.1 Data-Driven Approach

In the data-driven approach, machine learning models such as neural networks or recurrent architectures (e.g., BiLSTM) are trained directly on the dataset. The model learns a mapping from input time-series signals to output structural parameters.

Advantages:

- Capable of capturing complex nonlinear relationships
- Fast inference once trained
- Easy to implement and scale

Limitations:

- Requires large amounts of training data
- Lacks physical interpretability
- May fail to generalize outside training distribution

4.10.2 Physics-Informed Neural Networks (PINNs)

PINNs incorporate physical laws into the learning process by embedding the governing equations of motion into the loss function. Instead of relying purely on data, the model is guided by both data and physics.

Advantages:

- Enforces physical consistency
- Requires less data compared to purely data-driven models
- More robust to noise and unseen conditions

Limitations:

- Computationally more expensive
- Requires careful tuning of loss components
- Training can be more complex

4.10.3 Relevance to This Dataset

The dataset used in this study is particularly well-suited for both approaches:

- The presence of clean simulated data supports data-driven learning
- The availability of governing equations and parameters enables physics-based modeling
- The inclusion of measurement noise provides a realistic test for robustness

4.10.4 Key Insight

While data-driven models focus on learning patterns from data, PINNs ensure that the learned solution adheres to fundamental physical principles. Combining both approaches provides a comprehensive framework for structural health monitoring.

Chapter 5

Methodology and Implementation

5.1 Common Input Data for Stage 1 and Stage 2

Both Stage 1 and Stage 2 were implemented using the same synthetic 2DOF structural dynamics dataset. The shared input data consists of run-wise time-series responses, including time, dual excitation forces, displacement, velocity, acceleration, measured displacement signals, and the corresponding ground-truth physical and modal parameters. In the implementation, this common dataset is loaded through the shared `data_mdof.py` pipeline, where Stage 1 uses the displacement-based output format and Stage 2 extends the same loader to also return velocity and acceleration data when required.

5.2 Stage 1: Baseline PINN Framework

5.2.1 Overall Workflow

The first stage of the proposed methodology was designed as a **baseline parameter-identification framework** for a two-degree-of-freedom structural system using Physics-Informed Neural Networks. In this stage, the same shared 2DOF simulation dataset used throughout the project was processed on a **run-wise basis**, so that each simulation run could be treated as an independent inverse problem. The overall workflow consisted of loading one run from the dataset, extracting the corresponding time history and structural information, constructing a PINN model, enforcing the governing equations through a physics residual, training the model to reproduce the measured response while satisfying structural dynamics, and finally evaluating the identified parameters and reconstructed signals. This overall Stage 1 pipeline is reflected primarily in `trainer_stage1.py`, which serves as the central implementation script for training and organizing per-run outputs.

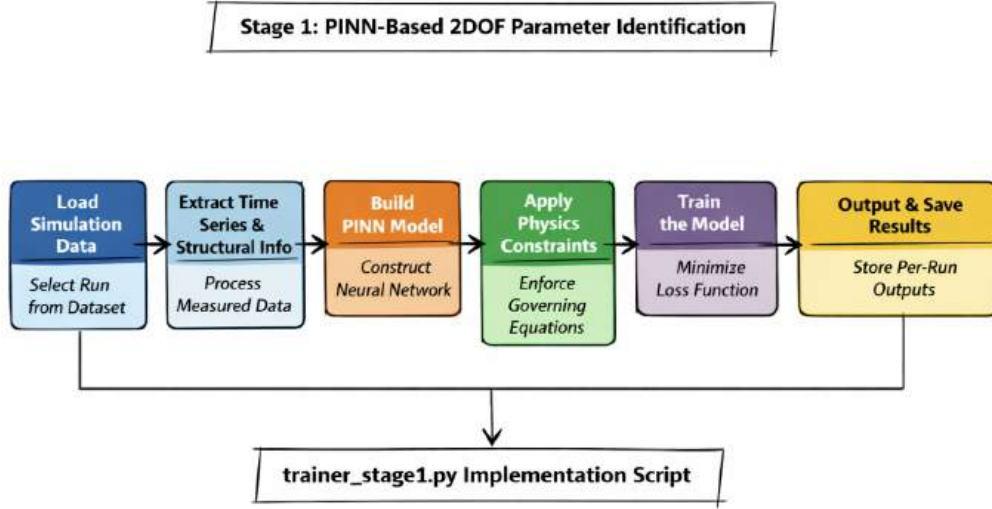


Figure 5.1: Baseline PINN Framework Overall Workflow

At the execution level, the Stage 1 workflow was controlled through a YAML configuration file, `config_mdof.yaml`, which specifies the dataset path, training settings, loss weights, perturbation level for initialization, and output directories. The configuration explicitly defines Stage 1 as a **baseline sanity-check setting** with dual excitation (F_1 and F_2 both active), a light parameter perturbation of $\pm 10\%$, no direct parameter supervision, and a small modal-loss term for semi-supervised guidance. This makes Stage 1 different from a fully blind identification setup, since it starts from physically reasonable initial guesses and allows limited modal support.

The actual launching of multiple runs was handled by `run_all_runs.py`, which delegates execution to the main training routine in `trainer_stage1.py`. This script makes it possible to train a selected range of run IDs using command-line arguments such as `-start`, `-end`, and `-force`. A secondary compatibility script, `train_one_run.py`, is also included in the codebase. Its role is not to implement a separate method, but rather to preserve backward compatibility by importing Stage 1 functions directly from `trainer_stage1.py`, so that older code using the earlier naming convention can still operate without modification.

Therefore, the overall Stage 1 workflow can be summarized as follows: configuration-driven setup, run-wise data loading, response and parameter extraction, collocation-point generation, baseline PINN initialization, training under data and physics constraints, saving of identified parameters and intermediate histories, and post-training evaluation with signal-, residual-, and parameter-level metrics. This structure forms the implementation backbone of the first methodological stage.

5.2.2 Data Generation / Data Loading

Stage 1 uses the common 2DOF synthetic dataset already described in the previous chapter. The implementation responsible for reading and organizing this dataset is contained in `data_mdof.py`. This script provides the data-handling utilities used by both Stage 1 and Stage 2, but in Stage 1 it is mainly used to load the complete CSV file, extract one simulation run at a time, and prepare the tensors required for PINN training. The function `load_full_csv()` reads the entire dataset into memory as a pandas DataFrame, while `get_all_run_ids()` identifies all unique RunID values so that the runs can be processed individually.

The central function for Stage 1 is `load_run_data()`. For a selected run ID, it extracts the time vector, the external excitation forces, the measured displacement signals, and the corresponding true structural and modal parameters. In its default Stage 1 usage, the function returns four objects: `t_data`, `F_data`, `u_meas`, and `true_par`. Here, `t_data` is the time tensor of shape $[N,1]$, `F_data` contains the dual input excitation forces $[F_1, F_2]$, `u_meas` contains the measured displacement responses of the two degrees of freedom, and `true_par` is a dictionary containing the physical parameters m_1, m_2, k_1, k_2 , the Rayleigh damping coefficients `alpha` and `beta`, and the modal targets `omega1`, `omega2`, `zeta1`, and `zeta2`.

An important implementation detail in `data_mdof.py` is the correction of the Rayleigh damping coefficient convention. The script explicitly notes that the original CSV stores the roles of `alpha` and `beta_ray` in a physically swapped form. To ensure consistency with the standard Rayleigh damping model

$$C = \alpha M + \beta K$$

the loader remaps the dataset columns so that the mass-proportional coefficient is assigned to `alpha` and the stiffness-proportional coefficient is assigned to `beta`. This correction is applied during loading so that all downstream scripts—including training, modal evaluation, and parameter reporting—work with the correct physical interpretation. This is a highly important methodological step because it guarantees that the implementation remains physically meaningful even though the raw data naming convention was inconsistent.

The data-loading script also includes the function `create_collocation_points()`, which generates the time-force pairs used for physics enforcement. If the requested number of collocation points exceeds the total number of available time steps, all points are used; otherwise, a random subset is selected on the same device. In this way, `data_mdof.py` performs not only raw data extraction but also the preparation of collocation samples required for the PINN loss. Thus, in Stage 1, this script serves as the interface between the simulation dataset and the physics-informed learning framework.

5.2.3 Preprocessing of Response Signals

In Stage 1, preprocessing was not implemented as a separate standalone script; instead, it was embedded within the data-loading and training pipeline. The main preprocessing tasks were carried out inside `data_mdof.py` and then consumed by `trainer_stage1.py`. The first preprocessing step was the selection of the correct response columns from the CSV file. The script uses a helper routine `_pick_cols()` to prioritize measured displacement columns such as `u1_meas` and `u2_meas`, while falling back to true signals if the measured versions are unavailable. This makes the loading process robust across different dataset variants. In the Stage 1 setting, the measured displacements are the primary observation signals used for data fitting.

A second preprocessing step involves the conversion of the selected arrays into PyTorch tensors located on the chosen computation device. The time vector is reshaped to $[N,1]$, the force matrix to $[N,2]$, and the measured displacement matrix to $[N,2]$. This conversion is necessary for automatic differentiation and GPU-based training. In addition, all quantities are cast to floating-point format suitable for neural network optimization. These operations are handled directly inside `load_run_data()`, thereby streamlining the data pipeline before training begins.

Although Stage 1 is primarily displacement-driven, the data utilities were designed to be backward compatible with later extensions. Specifically, `load_run_data()` can optionally return velocity and acceleration measurements when `return_va=True`. This is useful because the same script base supports both Stage 1 and Stage 2. However, in the Stage 1 baseline configuration defined in `config_mdof.yaml`, velocity and acceleration data loss are disabled (`use_va_loss: false`), indicating that the main fitted observation variable is displacement. Nevertheless, the training code itself includes support for optional `v` and `a` losses, which means the codebase was designed with future extensibility in mind.

Another form of preprocessing is the initialization of the inverse problem itself. Stage 1 does not begin from arbitrary parameter guesses; instead, it uses light perturbations around the true parameter values, with the perturbation level controlled through the configuration file. This serves as a practical baseline setup in which the parameter search begins near the correct physical solution. Such initialization can be considered part of the preprocessing pipeline, because it prepares the learning problem in a stable and interpretable manner before optimization. `trainer_stage1.py` includes helper routines for parameter perturbation and percentage-error tracking, while `train_one_run.py` exposes these same functions through backward-compatible imports.

Thus, although there is no separate “preprocessing script” by name, the Stage 1 preprocessing procedure consists of signal selection, tensor conversion, Rayleigh-coefficient correction, collocation-point preparation, and stable perturbed initialization. These steps

collectively ensure that the response data entering the PINN framework is both numerically usable and physically consistent.

5.2.4 PINN Architecture

The neural network architecture used in Stage 1 is defined in `model_mdof.py` through the class `TwoDOFPINN`. This model is a fully connected feed-forward neural network designed to learn the displacement response of a two-degree-of-freedom structural system as a function of time. The network input is a scalar time coordinate t , and the output is a two-dimensional displacement vector $[u_1(t), u_2(t)]$. The base architecture consists of three hidden layers with 64 hidden units each, followed by a final linear output layer producing the two displacement components. The hidden layers use the hyperbolic tangent activation function, which is a common choice in PINN implementations because it is smooth and supports stable differentiation.

In addition to the neural approximation of the displacement field, the model includes trainable physical parameters. Specifically, the structural masses m_1 and m_2 , the stiffness values k_1 and k_2 , and the Rayleigh damping coefficients are all embedded within the model. The damping coefficients are handled using a log-parameterization: the script stores base values `alpha0` and `beta0`, and learns multiplicative corrections via `alpha_log` and `beta_log`. The effective damping values are then obtained by exponentiating these log parameters. This design guarantees positivity of the damping coefficients and improves the stability of the inverse identification process.

A key architectural feature of `TwoDOFPINN` is that it does not only provide forward prediction of displacement. It also contains internal helper methods for automatic differentiation and for building the structural system matrices. The method `_d_dt()` computes derivatives of the network output with respect to time, thereby enabling the calculation of velocity and acceleration directly from the learned displacement field. The `_MCK()` method assembles the mass, damping, and stiffness matrices from the current trainable parameters. As a result, the network architecture integrates both a neural approximation component and the physical structural model in a single class.

The model also provides access to modal quantities such as natural frequencies and damping ratios, which are used later in the training and evaluation process. Since the model contains the current structural parameters, it can compute modal properties at any point during training. This is useful in Stage 1 because a small modal-loss component is permitted as a semi-supervised constraint. Therefore, the architecture is not limited to signal fitting alone; it is a complete inverse modeling framework capable of representing displacement trajectories, system matrices, and derived modal properties simultaneously.

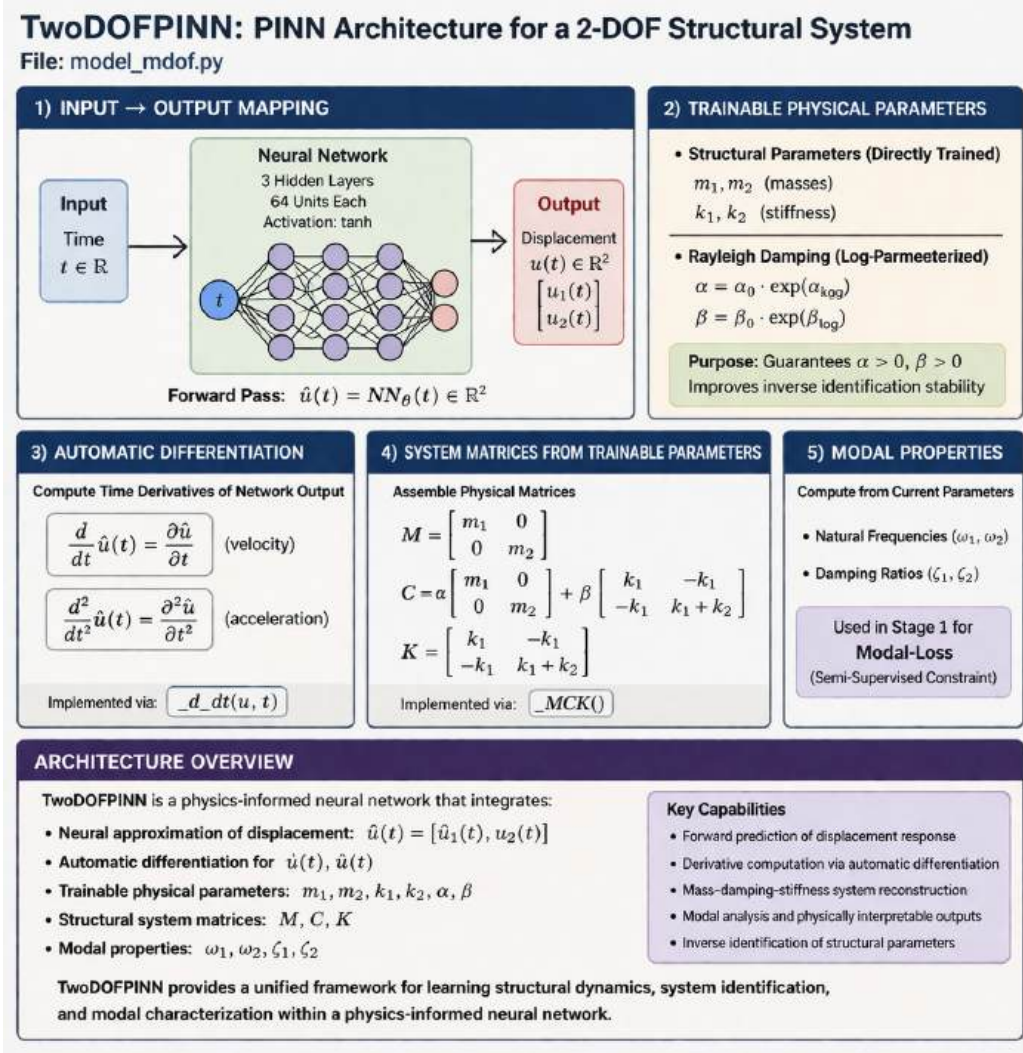


Figure 5.2: PINN Architecture Stage 1

Overall, `model_mdof.py` is the core physical-neural representation used in Stage 1. It defines how time is mapped to structural response, how derivatives are obtained through automatic differentiation, how the mass–damping–stiffness system is reconstructed from trainable parameters, and how physically interpretable quantities are extracted from the learned model. This script therefore forms the structural foundation of the baseline Stage 1 PINN.

5.2.5 Physics Residual Construction

The main physics-informed component of Stage 1 is the construction of the residual of the governing 2DOF equation of motion,

$$Mu''(t) + Cu'(t) + Ku(t) = F(t),$$

5.5 Physics Residual Construction

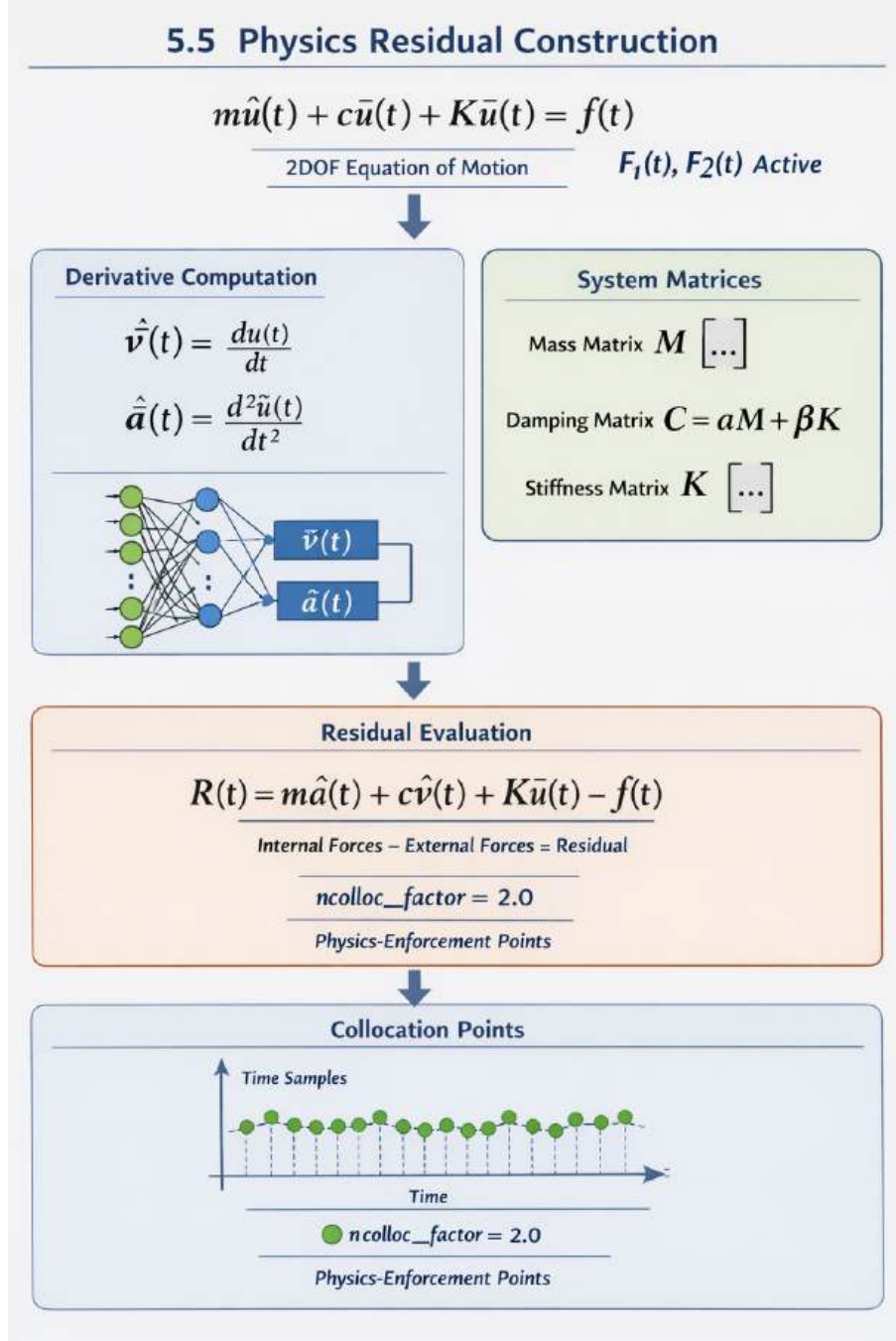


Figure 5.3: Physics Residual Construction Stage 1

with both excitation channels $F_1(t)$ and $F_2(t)$ active. This formulation is implemented within `model_mdof.py`, which provides the matrix construction routines and derivative operators needed to evaluate the residual at arbitrary collocation points. By computing the mismatch between the left-hand and right-hand sides of the structural dynamic equation, the model quantifies how far the neural prediction deviates from physically admissible motion.

The first part of residual construction is the generation of velocity and acceleration. Starting from the predicted displacement field $\hat{u}(t)$, the model differentiates once to obtain

$\hat{v}(t)$ and a second time to obtain $\hat{a}(t)$. These derivatives are computed via PyTorch automatic differentiation using the `_d_dt()` helper function in `model_mdof.py`. This makes the residual fully differentiable with respect to both network weights and physical parameters, which is essential for joint learning.

The second part is the construction of the system matrices. The mass matrix M , stiffness matrix K , and Rayleigh damping matrix C are built from the currently estimated parameters. The damping matrix follows the corrected convention $C = \alpha M + \beta K$, consistent with the remapping applied in `data_mdof.py`. Once M , C , and K are obtained, the residual can be evaluated as the difference between the modeled internal dynamic forces and the external forcing vector. Because both excitation inputs are active in Stage 1, the residual reflects the full coupled behavior of the 2DOF system under dual excitation.

The collocation points at which this residual is enforced are produced by `create_collocation_points` in `data_mdof.py`. The number of collocation points is controlled through `n_colloc_factor` in the configuration. In the uploaded Stage 1 configuration, this factor is set to 2.0, indicating that the number of physics-enforcement points is scaled relative to the number of available time samples. These collocation points are then passed into the training pipeline, where the residual is evaluated and used in the total loss function.

Thus, the physics residual in Stage 1 is not an external add-on; it is deeply integrated into the model and training structure. `model_mdof.py` provides the mathematical machinery, `data_mdof.py` provides the collocation samples, and `trainer_stage1.py` incorporates the resulting residual term into the optimization objective. Together, these components enforce structural dynamics directly during learning.

5.2.6 Training Procedure

The Stage 1 training process is implemented in `trainer_stage1.py`, which is the central script of the baseline methodology. This script supports both single-run and multi-run execution, but its main purpose is to automate the repeated training of the PINN for a specified range of simulation runs. It includes command-line arguments for selecting the run interval, re-running completed cases if necessary, loading a YAML configuration file, and organizing all result directories. Once executed, it loads the complete dataset, checks which runs have already been completed, and then iterates through the remaining runs one by one.

For each run, the trainer loads the corresponding time-series data and true parameters using `load_run_data()` from `data_mdof.py`. It then perturbs the initial structural parameters according to the configured perturbation level, instantiates the `TwoDOFPINN` model from `model_mdof.py`, and begins optimization. The configuration file `config_mdof.yaml` defines the Stage 1 hyperparameters: 3000 epochs, learning rate 10^{-3} , CUDA as the target device, and output directories under `results/mdof_pinn/stage1`. It also sets the key

loss weights for the baseline experiment: $w_{\text{data}} = 1.0$, $w_{\text{phys}} = 2.0$, $w_{\text{param}} = 0.0$, and $w_{\text{modal}} = 0.1$. These choices show that Stage 1 emphasizes data fitting and physics consistency, while avoiding direct supervision of the physical parameters themselves.

The training loop is designed to be robust and well instrumented. The script tracks parameter histories over epochs, saves intermediate metrics, and writes the trained model atomically to avoid corrupted checkpoint files. In addition, the script supports the generation of a large set of per-run plots, including detailed training history, parameter comparison, true-versus-predicted scatter plots, response comparison plots, power spectral density plots, and heatmaps of identification errors. This indicates that training was treated not simply as a black-box optimization process, but as a carefully monitored inverse modeling pipeline.

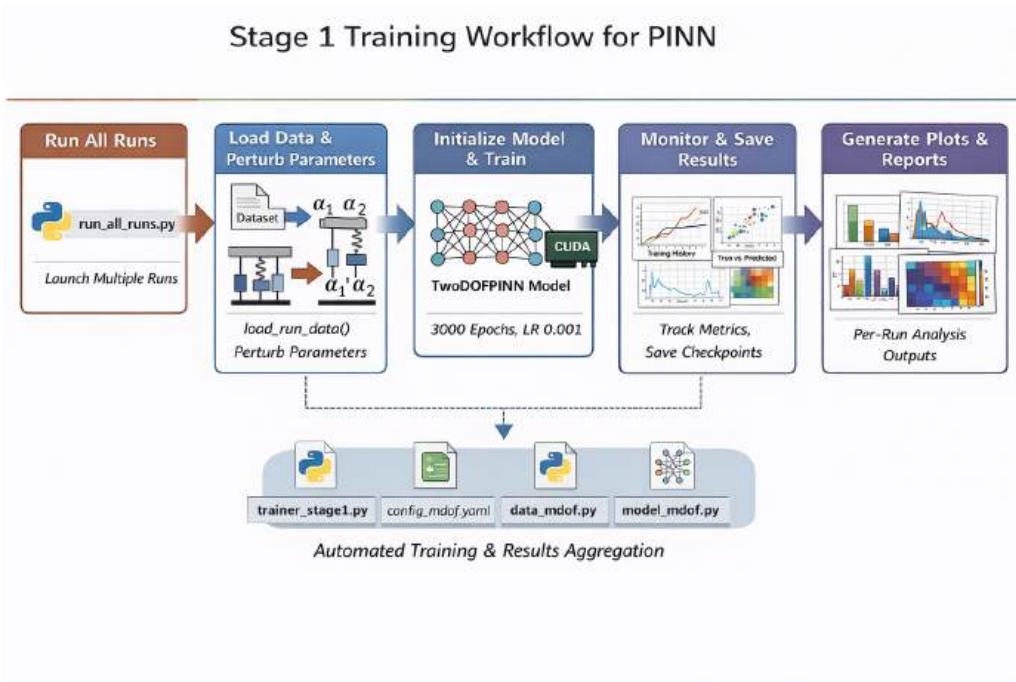


Figure 5.4: Training Procedure Stage 1

The Stage 1 trainer also supports optional advanced optimization settings through its built-in defaults, including warm-up epochs and LBFGS refinement. The code snippet shows a richer fallback configuration with warm-up and LBFGS enabled, while the uploaded YAML file focuses on the practical Stage 1 baseline settings. For scientific reporting, the most appropriate interpretation is that the Stage 1 framework was implemented with support for staged optimization and refinement, even though the primary run configuration is defined through `config_mdof.yaml`. The script architecture therefore provides flexibility for stable parameter identification across multiple runs.

At the workflow level, `run_all_runs.py` acts as the training launcher for multiple runs. It simply redirects execution to the `main()` routine in `trainer_stage1.py`, thereby

allowing the Stage 1 method to be executed over a chosen range of runs without manually invoking each one. In this sense, `run_all_runs.py` is part of the training procedure, but only as a job orchestration wrapper. Similarly, `train_one_run.py` does not implement a separate method; it only re-exports Stage 1 training functions for backward compatibility with older code.

In summary, the Stage 1 training procedure consists of run-wise model initialization, perturbed parameter setup, data-and-physics-constrained optimization, per-run artifact generation, and automatic aggregation of training outputs. The implementation is primarily concentrated in `trainer_stage1.py`, supported by `config_mdof.yaml`, `run_all_runs.py`, and the data/model utilities.

5.2.7 Parameter Identification Strategy

The purpose of Stage 1 is to solve an inverse structural dynamics problem: estimate the unknown structural parameters of the 2DOF system from measured dynamic responses. In this baseline setup, the parameters to be identified are the two masses (m_1, m_2), the two stiffness values (k_1, k_2), and the Rayleigh damping coefficients (α, β). In addition to these directly learned physical quantities, the model also reports derived modal properties, namely the natural frequencies (ω_1, ω_2) and damping ratios (ζ_1, ζ_2). This full parameter set is explicitly represented in both the data dictionary returned by `data_mdof.py` and the metadata definitions used in `trainer_stage1.py` and `evaluate_stage1.py`.

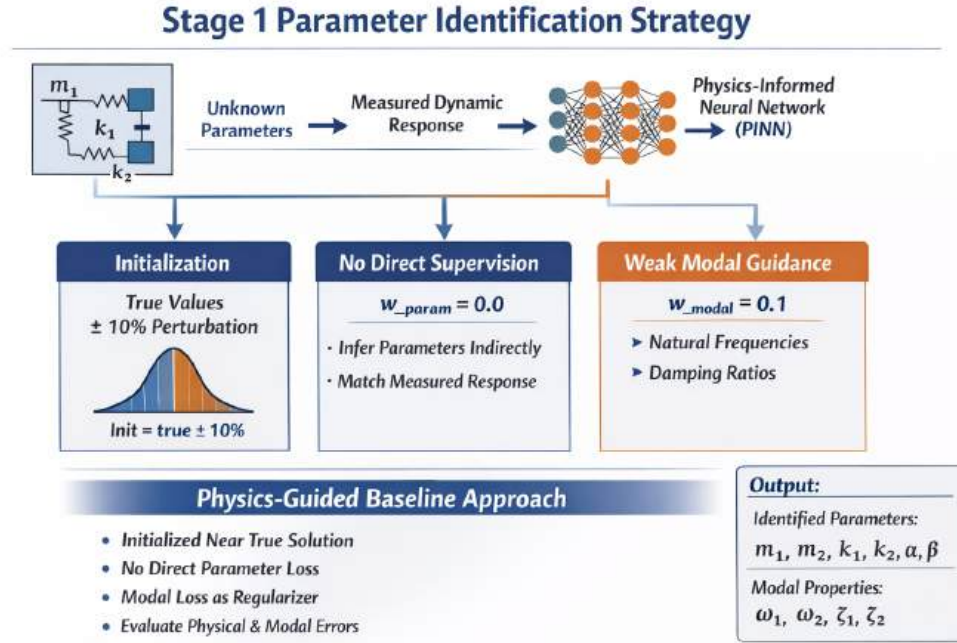


Figure 5.5: Parameter Identification Strategy Stage 1

The identification strategy in Stage 1 is deliberately designed as a baseline rather

than a fully blind inverse problem. The configuration file specifies that the model is initialized using the true parameters with a light random perturbation of 10%. This means the network does not start from arbitrary guesses; instead, it begins from physically plausible values near the solution. Such a setup is appropriate for a first-stage sanity check because it allows the implementation to verify whether the PINN can recover the correct parameter set under controlled conditions before moving to a more difficult blind setting in Stage 2. The Stage 1 notes in `config_mdof.yaml` explicitly describe this as a baseline sanity-check stage with `init = true ± 10%`.

Another important feature of the Stage 1 identification strategy is the use of **no direct parameter supervision**. The uploaded configuration sets `w_param = 0.0`, meaning that the true values of m, k, α, β are not directly inserted into the loss as regression targets. Instead, the model is expected to infer these parameters indirectly by matching the measured response and satisfying the governing equation. At the same time, a small modal-loss term `w_modal = 0.1` is allowed. This means that Stage 1 is not completely unsupervised in a physical sense, because modal guidance is used as a weak structural regularizer. This design provides a compromise between pure inverse identification and stability-oriented supervision.

The resulting identified parameters are saved on a per-run basis during training. `trainer_stage1.py` generates metrics tables, parameter histories, and comparison plots that include the true, initial, and predicted values. It also aggregates all identified parameters into a global summary CSV and produces a Markdown report of the results. The script `summarize_results.py` provides an additional post-processing utility for aggregating parameter-identification results across runs, extracting summary statistics, and detecting outliers. Although it is not the core training script, it contributes to the interpretation of the identification performance at the dataset level.

Therefore, the Stage 1 parameter identification strategy can be characterized as a **physics-guided baseline inverse learning approach**: the parameters are trainable within the PINN, initialized near the true solution, learned without direct parameter regression loss, weakly regularized through modal information, and evaluated using both physical and modal error measures. This makes Stage 1 an important intermediate step between theoretical PINN formulation and the more challenging fully blind setup introduced later.

5.2.8 Evaluation and Visualization

The evaluation of Stage 1 was performed after training using `evaluate_stage1.py`. This script reloads the saved model for each run, reconstructs the predicted displacement, velocity, and acceleration signals, computes the physics residual at collocation points, and compares the identified parameters against their reference values. It therefore serves

as the main post-training analysis tool for the baseline framework. The evaluation script again depends on `data_mdof.py` for loading run-level data and on `model_mdof.py` for forward prediction and residual computation, thereby maintaining consistency with the training pipeline.

At the signal level, `evaluate_stage1.py` computes standard regression metrics for the displacement, velocity, and acceleration signals of both degrees of freedom. The per-run output file `eval_signal_metrics.csv` contains quantities such as mean squared error, root mean squared error, mean absolute error, coefficient of determination (R^2), and normalized root mean squared error. These metrics allow the signal reconstruction ability of the PINN to be quantified beyond parameter recovery alone. In addition, the script computes residual statistics such as mean absolute residual, RMS residual, maximum residual, and standard deviation, and stores them in `eval_residuals.csv`. This makes it possible to assess not only data fit but also the degree to which the learned solution satisfies the structural dynamic equations.

At the parameter level, the evaluation script reads the saved metrics from the run directory and generates `eval_param_metrics.csv`, which contains the true, predicted, and percentage-error values for all physical and modal parameters. These results are then visualized through multiple per-run plots, including signal-metric bar charts, residual waveforms, true-versus-predicted parameter grids, and evaluation (R^2) heatmaps. In addition to per-run outputs, the script aggregates all successful evaluations into a global summary table and creates dashboard plots across runs, such as grouped plots, grouped NRMSE plots, physics-residual plots, and threshold-based parameter-accuracy plots. Finally, a Markdown evaluation report is written at the stage root.

The training-side script `trainer_stage1.py` also contributes substantially to visualization. During training it creates detailed per-run plots such as training-history panels, parameter-comparison charts, signal response overlays, Welch PSD comparisons with natural-frequency markers, and heatmaps of parameter errors. After multiple runs are completed, it additionally generates dashboard plots summarizing error behavior and final loss values across runs. Thus, visualization in Stage 1 is distributed across both training and evaluation: the trainer monitors optimization and identification behavior, while the evaluator provides formal post-training performance assessment.

A final supporting role is played by `summarize_results.py`, which provides a utility for aggregating identified-parameter results across runs and extracting summary statistics and outlier information. This script is not essential for the core evaluation pipeline, but it strengthens the reporting workflow by making it easier to study overall identification trends rather than only isolated run-level results.

In conclusion, the Stage 1 evaluation and visualization framework is comprehensive. It examines the learned model from multiple perspectives: signal reconstruction accuracy, physics-consistency residuals, physical-parameter errors, modal-parameter errors, per-

run plots, multi-run dashboard plots, and textual summary reports. This makes the baseline stage not only a training exercise, but also a fully documented and interpretable parameter-identification workflow.

5.3 Stage 2: Fully Blind PINN Framework

5.3.1 Overall Workflow

Stage 2 extends the baseline framework developed in Stage 1 into a **more challenging fully blind inverse identification setting**. While both stages use the same underlying 2DOF synthetic dataset, Stage 2 removes the limited semi-supervision allowed in Stage 1 and instead relies on a stricter physics-informed learning strategy. In this stage, the objective is to identify the structural parameters of the two-degree-of-freedom system using only the measured dynamic responses and the governing equations of motion, without direct parameter supervision and without modal supervision. This design is explicitly stated in the Stage 2 trainer, which describes the method as a “fully-blind 2DOF PINN” with no modal or parameter supervision.

The overall workflow of Stage 2 follows the same broad sequence as Stage 1, but with a more advanced optimization strategy. First, the shared dataset is loaded run by run through the common `data_mdof.py` pipeline. For each selected run, the required time-series signals and true reference parameters are extracted. Next, the PINN model is instantiated using a scaled-parameter version of the base 2DOF PINN architecture. The model is then trained in two phases: an initial warm-up phase focused only on data fitting, followed by a gradual introduction of the physics loss using a smooth sigmoid ramp. During this second phase, the trainable physical parameters are unfrozen and optimized jointly with the neural network weights. After training, the results are saved in dedicated run folders, followed by post-training evaluation, plotting, and result summarization.

Stage 2: Fully-Blind 2DOF PINN Workflow

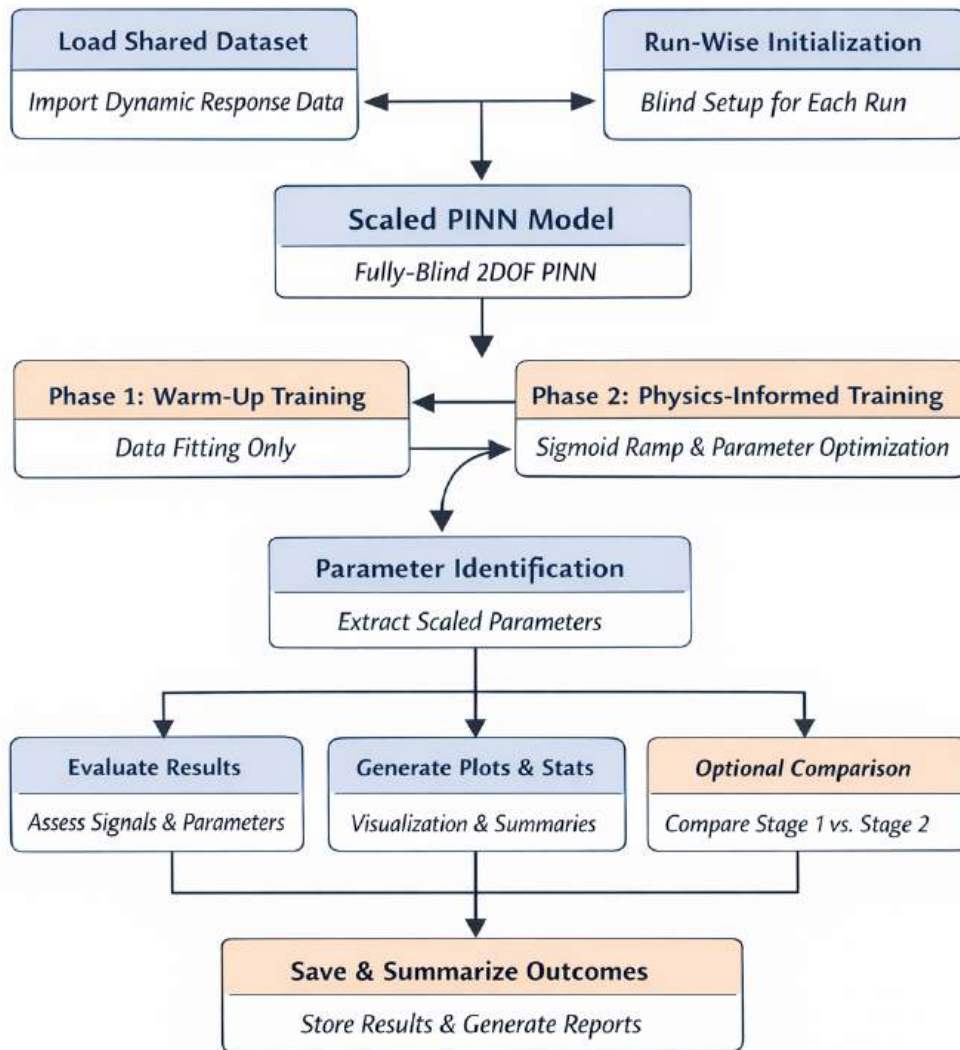


Figure 5.6: Fully Blind PINN Framework Overall Workflow Stage 2

The implementation is centered around `trainer_stage2.py`, which contains the complete Stage 2 training logic, model scaling strategy, optimization settings, plotting routines, and output management. The post-training quantitative analysis is handled by `evaluate_stage2.py`, while `plot_stage2.py`, `summarize_stage2_results.py`, and `export_stage2_t`

support visualization, statistical summarization, and thesis-table generation. An additional script, `compare_stage1_stage2.py`, is included to compare the performance of the Stage 1 and Stage 2 frameworks after both have been completed. However, this comparison script is not part of the core Stage 2 methodology itself; rather, it supports later interpretation of the results.

Thus, the Stage 2 workflow can be summarized as follows: shared data loading, run-wise blind initialization, scaled PINN construction, staged optimization with warm-up and physics ramping, parameter identification through scale factors, evaluation of signals and identified parameters, generation of plots and summary statistics, and optional comparison against the Stage 1 baseline. This stage therefore represents the final and more demanding implementation of the proposed inverse PINN framework.

5.3.2 Data Generation / Data Loading

Stage 2 uses the **same 2DOF synthetic dataset** employed in Stage 1. This is consistent across the Stage 2 scripts: both the trainer and evaluator point to the same dataset path, `Data/mdof_2dof_216runs_pinn.csv`, confirming that Stage 2 does not use a separate input source. The Stage 2 configuration file also references this same dataset, which supports the statement that both stages are based on an identical input foundation.

The data-loading logic itself is not reimplemented separately for Stage 2; instead, the framework reuses the common `data_mdof.py` utilities already established earlier. This is an important methodological choice, because it ensures that differences between Stage 1 and Stage 2 arise from the identification strategy and optimization design rather than from inconsistent data handling. In the Stage 2 case, the shared loader is used in its extended form with `return_va=True`, so that the training pipeline receives not only measured displacement signals but also velocity and acceleration signals. This can be seen directly in the Stage 2 evaluation workflow, where `load_run_data()` returns six outputs: time, force, measured displacement, measured velocity, measured acceleration, and the dictionary of true parameters.

For each selected run, the extracted input data therefore includes the time vector t , the dual excitation force matrix $F = [F_1, F_2]$, the measured displacement signals (u_1, u_2) , the measured velocity signals (v_1, v_2) , the measured acceleration signals (a_1, a_2) , and the true run-level parameters used only for later evaluation. Compared with Stage 1, the inclusion of velocity and acceleration signals means that the Stage 2 data pipeline supports richer dynamic supervision. This is one of the key differences between the two stages, even though the raw dataset itself remains unchanged.

As in Stage 1, collocation points for physics enforcement are also generated from the same time-force data. The number of collocation points is controlled by `n_colloc_factor`; in the main Stage 2 trainer configuration this factor is set to 4.0, which is larger than

the older auxiliary YAML file and indicates a denser physics-enforcement strategy in the actual final implementation. These collocation points are used during training to evaluate the residual of the governing equations separately from the direct data-fitting samples.

Therefore, the Stage 2 data-loading process is best described as an **extended reuse of the shared Stage 1 data pipeline**. The dataset itself is unchanged, but the Stage 2 implementation makes fuller use of the available dynamic information by including displacement, velocity, acceleration, and collocation-based physics data in the inverse learning process.

5.3.3 Preprocessing of Response Signals

The preprocessing strategy in Stage 2 is also based on the common data pipeline, but it becomes more significant because the training objective now includes multiple dynamic response components. The first preprocessing step is the extraction of the required signals from the shared run-wise dataset. Using the extended output mode of the data loader, Stage 2 obtains measured displacement, velocity, and acceleration signals alongside the input forces and time vector. This means that the data entering the Stage 2 PINN is no longer limited to displacement-only fitting, but instead supports a multi-response learning setup.

The second preprocessing step involves conversion into PyTorch tensors on the chosen computation device. The Stage 2 trainer supports both CPU and CUDA execution, with the actual device automatically selected based on hardware availability. The time vector is reshaped appropriately for neural network input, while the force and signal arrays are maintained in matrix form to preserve the two-degree-of-freedom structure of the problem. These tensors are then used directly in both the data loss and the physics loss.

A third and particularly important preprocessing element in Stage 2 is the generation of **blind initial parameter guesses**. The script `trainer_stage2.py` defines a function `perturb_params()` that perturbs the true masses, stiffnesses, and Rayleigh damping coefficients within a specified range. In the main trainer configuration, the perturbation level is set to 10%, whereas an older standalone YAML file shows an earlier version with 15% perturbation. For scientific reporting, the actual implemented trainer logic is the more appropriate reference. This perturbation process creates the initial guesses from which blind inverse learning begins. Unlike direct initialization at the exact true values, this approach requires the model to recover the correct parameters from nearby but imperfect starting points.

Another preprocessing-related operation is the preparation of the collocation data used for physics enforcement. The collocation samples are generated from the time-force history and then transferred to the same device as the model. These samples are distinct from the data-fitting samples and are used only to evaluate the governing-equation resid-

ual. This separation between supervised response points and physics-enforcement points is a core feature of PINN training.

Thus, the preprocessing in Stage 2 consists of extracting a richer set of response signals, creating blind perturbed initial parameters, converting all quantities into device-ready tensors, and preparing collocation points for residual enforcement. These steps ensure that the Stage 2 training process operates on a numerically consistent and physically meaningful representation of the shared structural dynamics dataset.

5.3.4 PINN Architecture

The Stage 2 model architecture is built on top of the same base `TwoDOFPINN` class used in Stage 1, but it introduces an important modification for parameter identification. Instead of directly learning the masses and stiffnesses as raw trainable parameters, Stage 2 defines a subclass called `ScaledTwoDOFPINN`, in which m_1, m_2, k_1 , and k_2 are represented through **scale factors** applied to fixed reference values. This reference values correspond to the blind initial guesses generated at the start of the run. The scale parameters are initialized to 1.0, so that the effective initial mass and stiffness values match the chosen perturbed references.

This scaled representation is introduced to improve optimization of conditioning. Since the masses and stiffnesses may have very different absolute magnitudes, directly optimizing them as raw parameters can be numerically unstable. By rewriting them as

$$m_i = m_{i,\text{ref}} \cdot s_{m_i}, \quad k_i = k_{i,\text{ref}} \cdot s_{k_i},$$

the optimization variables become dimensionless scale factors centered near unity. This makes the inverse problem easier to optimize and allows the training routine to constrain these scales to a physically reasonable range. In fact, the trainer includes an explicit `_clamp_scales()` function that restricts the scale parameters to the interval $[0.5, 2.0]$ optimization.

Figure 5.1: ScaledTwoDOFPINN Architecture

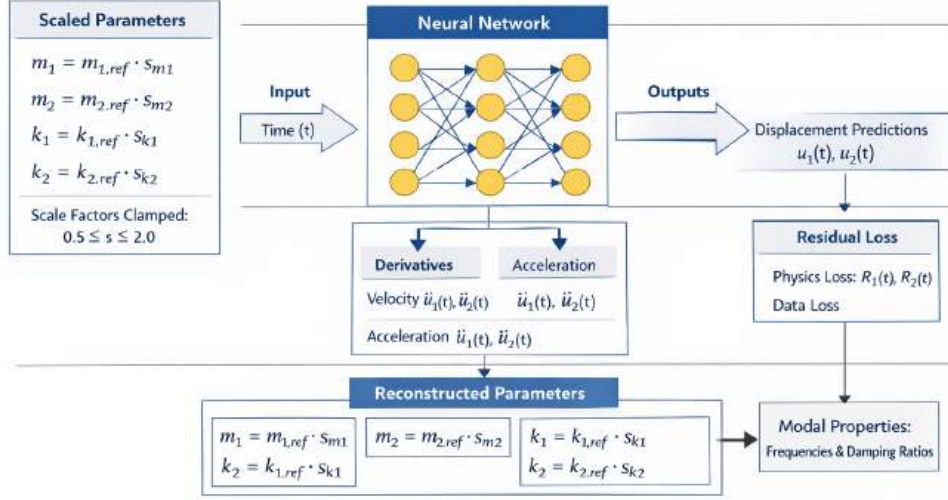


Figure 5.7: PINN Architecture Stage 2

The neural component of the architecture remains the same as in the base model: time is mapped to the two displacement responses through a fully connected neural network. Velocity and acceleration are then obtained through automatic differentiation. The underlying mass, damping, and stiffness matrices are reconstructed from the current trainable parameters, and the model can compute derived modal properties such as natural frequencies and damping ratios. In this way, the Stage 2 PINN retains the physically interpretable structure of the Stage 1 model while using a more stable parameterization for inverse learning.

Another architectural component concerns the damping parameters. The Stage 2 code retains the log-parameterized treatment of Rayleigh damping inherited from the base model. The helper functions `_get_damp_params()` and `_alpha_beta_vals()` in `trainer_stage2.py` show that the trainer accesses these damping terms through model-specific utilities rather than assuming a single hard-coded parameter form. This keeps the Stage 2 architecture compatible with the physically constrained damping representation used throughout the project.

Overall, the Stage 2 architecture can be described as a **scaled-parameter extension of the base 2DOF PINN**. Its main novelty is not a different neural network topology, but a more robust parameter-identification formulation in which the unknown masses and stiffnesses are learned through bounded dimensionless scale factors relative to blind initial references. This modification is central to the success of the fully blind Stage 2 inverse framework.

5.3.5 Physics Residual Construction

The physics residual in Stage 2 is based on the same 2DOF governing equation of motion as in Stage 1,

$$Mu''(t) + Cu'(t) + Ku(t) = F(t),$$

but its treatment in the optimization process is more advanced. As before, the model predicts displacement as a function of time, and velocity and acceleration are obtained by automatic differentiation. The mass, damping, and stiffness matrices are built from the current parameter estimates, and the residual is evaluated at collocation points generated from the run-specific time-force data. The model method `compute_residuals()` is called within the trainer to compute this physics inconsistency term.

The key improvement in Stage 2 lies in **residual normalization**. The trainer defines a helper function `_normalized_residual_mse()` that divides the residual by its mean absolute magnitude before squaring and averaging it. This normalization is introduced for stability, so that the physics loss does not dominate training merely because the raw residual values are numerically large. The use of this normalized residual mean-squared error is explicitly described in the Stage 2 trainer and is an important distinction from simpler PINN implementations that use an unscaled residual term.

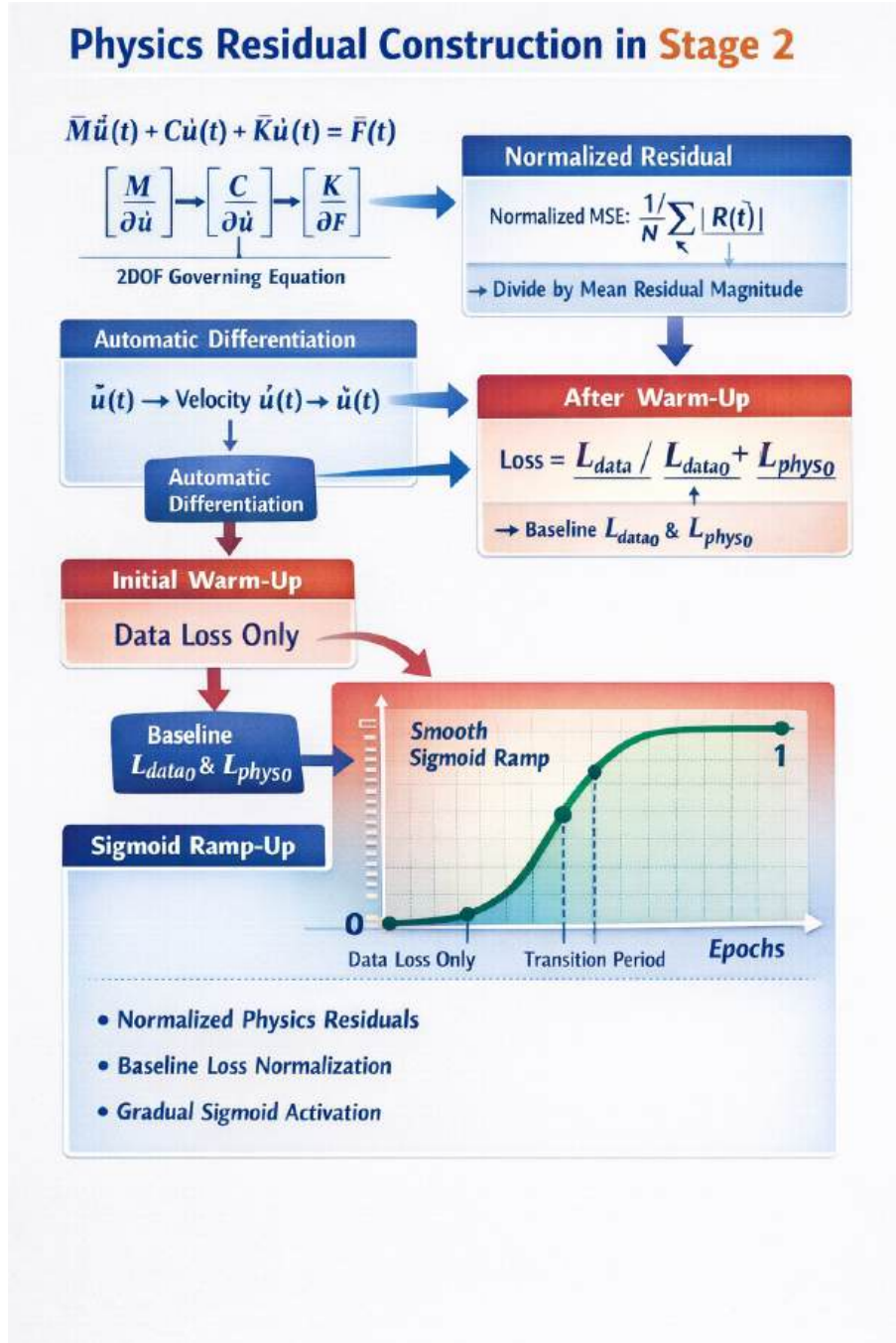


Figure 5.8: Physics Residual Construction Stage 2

A second major improvement is the **staged integration of the physics loss**. During the initial warm-up period, only the data loss is optimized and the physical parameters remain frozen. After this warm-up ends, the trainer computes baseline magnitudes for the data loss and physics loss, denoted in the code as `Ldata0` and `Lphys0`. These baseline values are then used to normalize the two loss components when forming the total objective. This prevents one component from overwhelming the other and gives the ramping strategy a more controlled effect.

The actual weighting of the physics term after warm-up is governed by a **smooth**

sigmoid ramp, implemented through `_sigmoid_ramp()`. Rather than switching the physics loss on abruptly, the trainer increases its influence gradually over a specified number of ramp epochs. This is a valuable methodological design choice because it avoids destabilizing the network before a reasonable response reconstruction has been learned. By the time the physics residual becomes fully active, the neural network already has a good data-based initialization of the dynamic trajectories.

In summary, Stage 2 uses the same physical governing equations as Stage 1, but with a more sophisticated residual treatment: normalized physics residuals, baseline-based loss normalization, and sigmoid-ramped activation after warm-up. These improvements make the physics-enforcement mechanism significantly more stable and more suitable for a blind inverse identification task.

5.3.6 Training Procedure

The training procedure of Stage 2 is implemented entirely in `trainer_stage2.py`. This script supports run-wise processing of the shared dataset and stores the outputs in a dedicated Stage 2 results structure under `results/mdof_pinn/stage2_fixed/`. The training configuration built directly into the script defines 4000 epochs, warm-up and ramp periods, gradient clipping, parameter-specific learning rates, optional LBFGS refinement, and a `ReduceLROnPlateau` scheduler. These settings represent the final advanced version of the Stage 2 pipeline and are more detailed than the older auxiliary YAML file.

For each run, the trainer first loads the required time, force, displacement, velocity, and acceleration data, along with the corresponding true parameter dictionary. It then perturbs the true physical parameters to create blind initial guesses and instantiates the scaled Stage 2 PINN model from these references. Before optimization begins, the scale parameters for the masses and stiffnesses, as well as the damping parameters, are frozen so that the network first focuses on learning the dynamic response itself. This warm-up phase lasts for a user-defined number of epochs, which is 800 in the main Stage 2 configuration.

During the warm-up phase, the loss consists only of the data term, which in Stage 2 is itself multi-component:

$$L_{\text{data}} = w_u L_u + w_v L_v + w_a L_a.$$

The default trainer weights are $w_u = 2.0$, $w_v = 0.5$, and $w_a = 1.5$ showing that displacement remains the dominant signal but velocity and acceleration are also used to strengthen dynamic supervision. This is a major difference from Stage 1, where the baseline configuration largely centered on displacement-based fitting.

At the end of the warm-up phase, the trainer unfreezes the physical parameters and records baseline loss magnitudes. From that point onward, the physics loss is gradually

introduced through the sigmoid ramp. The total loss after warm-up is given by the normalized data loss, the ramp-weighted normalized physics loss, and a small scale-prior term:

$$L = \frac{L_{\text{data}}}{L_{\text{data},0}} + w_{\text{phys}} r(e) \frac{L_{\text{phys}}}{L_{\text{phys},0}} + \lambda_{\text{scale}} L_{\text{prior}}.$$

The scale-prior term penalizes unnecessary deviation of the mass and stiffness scale factors from 1.0. This acts as a regularizer that discourages unrealistic parameter drift while still allowing the model to move away from the initial blind guess when necessary.

Optimization is performed primarily with Adam, but the parameter groups are assigned different learning rates: one for the neural network weights, one for stiffness scales, one for mass scales, and one for damping parameters. This is an important implementation detail because different parameter types have different sensitivities and magnitudes. The trainer also applies gradient clipping with a norm limit of 5.0 and uses a `ReduceLROnPlateau` scheduler to reduce the learning rate when progress stalls. Finally, optional LBFGS refinement is supported after the Adam phase to improve convergence in the late optimization stage. These features collectively make Stage 2 considerably more robust than a standard single-optimizer PINN training loop.

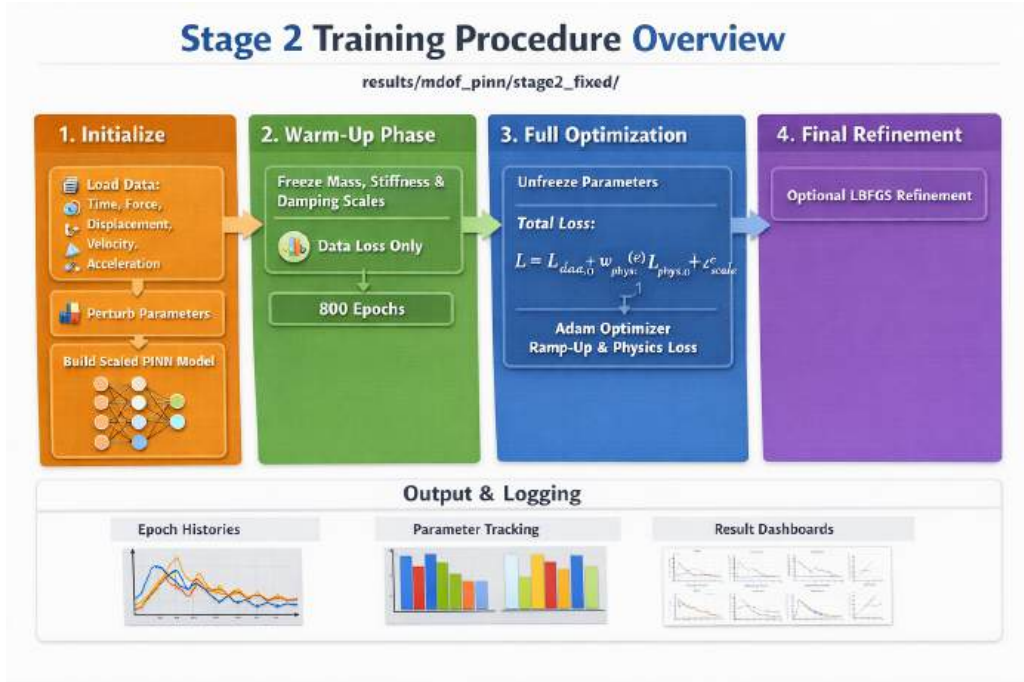


Figure 5.9: Training procedure of Stage 2

The trainer records epoch-wise histories, parameter histories, and run-level outputs. It also generates per-run plots and a combined dashboard at the end of multi-run training. Consequently, the Stage 2 training procedure is not only a numerical optimization routine but a carefully structured blind inverse-learning workflow with staged activation,

normalized objectives, differentiated learning rates, and extensive result logging.

5.3.7 Parameter Identification Strategy

The Stage 2 parameter identification strategy is explicitly designed to be **fully blind**. Unlike Stage 1, it does not use modal supervision and does not use direct parameter supervision. This is stated clearly in the trainer description and reinforced by the configuration values `w_modal = 0.0` and `w_param = 0.0`. Thus, the physical parameters must be inferred solely by reproducing the measured structural response and satisfying the governing equations of motion.

The parameters identified in Stage 2 are the same core physical quantities as before: masses m_1 and m_2 , stiffnesses k_1 and k_2 , and the Rayleigh damping coefficients α, β . In addition, the framework evaluates the derived modal quantities $\omega_1, \omega_2, \zeta_1$, and ζ_2 .

However, unlike Stage 1, the masses and stiffnesses are not trained directly as raw variables. Instead, they are represented through bounded scale factors applied to blind initial references. This makes the identification process more stable and also more interpretable, because the final estimates can be viewed as corrections to the perturbed starting values.

Another important aspect of the Stage 2 strategy is that the physical parameters are not optimized from the very beginning. During the warm-up phase, they remain frozen, allowing the neural network to first learn the overall response manifold. Only after the warm-up ends are the parameters unfrozen and the blind identification process begins in earnest. This staged approach reduces the risk of parameter collapse or unstable early updates. In addition, the scale-prior regularization helps keep the identified masses and stiffnesses within a plausible neighborhood of the initial blind references unless the data and physics terms provide strong evidence for a different value.

5.7 Parameter Identification Strategy

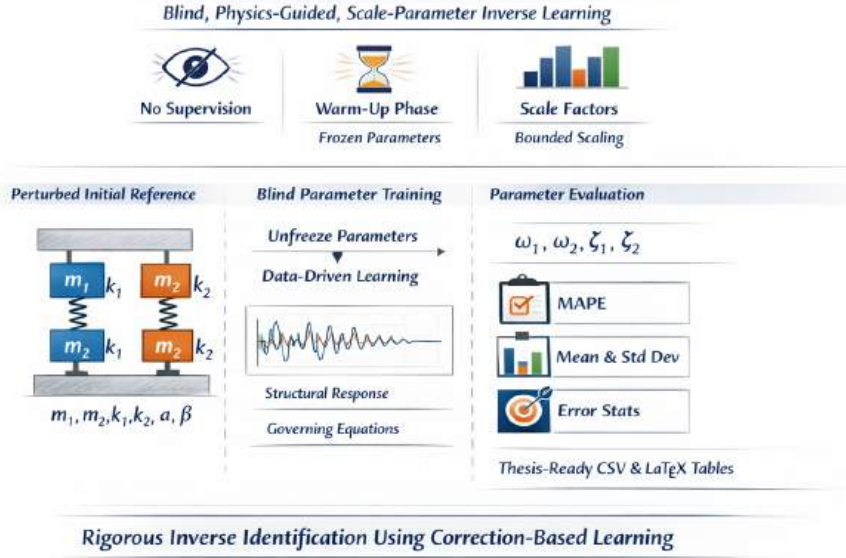


Figure 5.10: Stage 2 parameter identification strategy

The identified parameter results are written into per-run metrics files and a global summary CSV. These outputs are later consumed by the evaluation and summarization scripts. `summarize_stage2_results.py` computes MAPE, mean, standard deviation, median, maximum error, and success-rate statistics for the identified parameters, while `export_stage2_tables.py` converts the final Stage 2 summary results into thesis-ready CSV and LaTeX tables. These scripts show that the identification strategy was designed not only for optimization but also for systematic post-hoc interpretation across multiple runs.

Accordingly, the Stage 2 identification strategy can be described as a **blind, physics-guided, scale-parameter inverse learning approach**. It estimates the structural parameters without direct supervision, uses bounded multiplicative corrections relative to perturbed initial guesses, activates parameter learning only after a data-based warm-up, and evaluates identification quality using both physical and modal quantities. This makes Stage 2 the most rigorous inverse identification setting in the project.

5.3.8 Evaluation and Visualization

The quantitative evaluation of Stage 2 is performed by `evaluate_stage2.py`. This script reloads the saved Stage 2 model for each run, reconstructs the predicted displacement, velocity, and acceleration responses, evaluates the physics residual, and computes parameter-error metrics. The script then stores these outputs both at the run level and at the global stage level. It therefore functions as the formal evaluation counterpart to

`trainer_stage2.py`.

At the signal level, the evaluator computes standard regression metrics for u , v , and a across both degrees of freedom. These include mean squared error, root mean squared error, mean absolute error, coefficient of determination (R^2), and normalized root mean squared error. The results are written to `eval_signal_metrics.csv` for each run. At the physics level, the residuals are evaluated at collocation points and summarized in `eval_residuals.csv` using statistics such as mean absolute residual, RMS residual, maximum residual, and standard deviation. This makes it possible to judge both how well the learned trajectories match the observed signals and how well they satisfy the governing equations.

At the parameter level, the evaluator reads the saved metrics from each run and generates `eval_param_metrics.csv`, which contains the true, predicted, and percentage-error values for all physical and modal parameters. It also computes summary indicators such as how many parameters fall within 5% or 10% error and the mean parameter error percentage across the run. These statistics are included in the automatically generated Stage 2 evaluation report, which is written as `evaluation_report.md`. The report contains structured sections for signal reconstruction, NRMSE, physics residuals, and parameter accuracy.

Visualization is supported through both the evaluator and the dedicated plotting script `plot_stage2.py`. The evaluator creates per-run plots such as signal-metric charts, residual waveforms, true-versus-predicted parameter plots, and evaluation heatmaps, and also generates global plots such as grouped (R^2) charts, grouped NRMSE charts, physics residual plots, and parameter-threshold plots. `plot_stage2.py` extends this capability by generating a broader range of reusable result figures from saved CSVs and histories, including parameter identification plots, modal plots, training-history plots, run-ranking plots, heatmaps, and improvement plots. Because it works directly from saved summaries and histories, it provides a lightweight and reproducible way to prepare final result figures without rerunning training.

Two additional support scripts strengthen the final reporting workflow. `summarize_stage2_results` aggregates Stage 2 identification performance into summary statistics and outlier tables, while `export_stage2_tables.py` exports compact thesis tables in CSV and LaTeX formats. Finally, `compare_stage1_stage2.py` enables a direct comparison of Stage 1 and Stage 2 parameter-identification statistics by computing stage-wise MAPE summaries and producing a comparison plot. This comparison script is especially useful for later discussion chapters where the progression from the baseline framework to the fully blind framework is analyzed.

5.4 Summary of Stage 2

Stage 2 extended the baseline PINN framework into a fully blind inverse identification setting by removing direct supervision, introducing scaled parameter learning, stabilizing optimization through warm-up and sigmoid-ramped physics enforcement, and using a more comprehensive evaluation and reporting pipeline. Together, Stage 1 and Stage 2 provide a progressive methodological development from controlled baseline identification to robust blind parameter estimation.

Chapter 6

Results and Discussion

6.1 Overview of Results Stage 1: Baseline PINN Framework

Stage 1 results are reported for **5 evaluated runs**. The results below are organized by subsection using the combined Stage 1 training and evaluation summaries.

6.2 Predicted vs True Responses

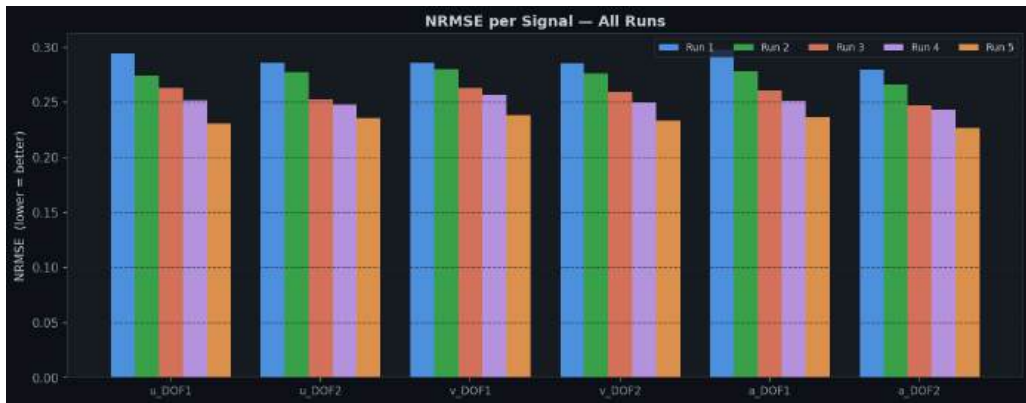


Figure 6.1: NRMSE per Signal – All Runs Stage 1

6.2.1 Signal Reconstruction (R^2) for all Stage 1 runs

Run	u_DOF 1	u_DOF 2	v_DOF 1	v_DOF 2	a_DOF 1	a_DOF 2	Mean R^2
1	0.0000	-0.0002	-0.0000	-0.0000	-0.0003	-0.0003	-0.0001
2	-0.0005	-0.0003	-0.0000	-0.0000	-0.0003	-0.0003	-0.0003
3	-0.0004	-0.0005	-0.0000	-0.0000	-0.0003	-0.0003	-0.0003
4	-0.0002	-0.0004	-0.0001	-0.0001	-0.0003	-0.0003	-0.0002
5	-0.0002	-0.0002	-0.0002	-0.0002	-0.0002	-0.0002	-0.0002

Figure 6.2: Signal Reconstruction (R^2) Stage 1

6.2.2 Signal Reconstruction NRMSE for all Stage 1 runs

Run	u_DOF 1	u_DOF 2	v_DOF 1	v_DOF 2	a_DOF 1	a_DOF 2	Mean
1	0.29438	0.28618	0.28601	0.28528	0.29761	0.27936	0.28814
2	0.27420	0.27719	0.28029	0.27630	0.27839	0.26617	0.27542
3	0.26307	0.25283	0.26320	0.25903	0.26094	0.24761	0.25778
4	0.25163	0.24807	0.25686	0.25014	0.25115	0.24304	0.25015
5	0.23085	0.23573	0.23824	0.23370	0.23689	0.22677	0.23370

Figure 6.3: Signal Reconstruction NRMSE for all Stage 1

6.2.3 Discussion

The Stage 1 response reconstruction remained weak across all five runs. The mean (R^2) stayed near zero or slightly negative in every case, while mean NRMSE improved gradually from **0.28814** in Run 1 to **0.23370** in Run 5. This indicates some reduction in normalized signal mismatch, but not strong time-domain agreement between measured and predicted responses.

6.2.4 Estimated Structural Parameters



Figure 6.4: Parameter % Error All Stage 1 runs

6.2.5 Detailed Stage 1 parameter-identification results for all runs

Stage 1 — Mass Parameters (m_1, m_2)								
Run	m1 True	m1 Init	m1 Pred	m1 Err%	m2 True	m2 Init	m2 Pred	m2 Err%
1	800.0	768.997	769.246	3.844%	1600.0	1524.820	1525.069	4.683%
2	800.0	740.638	740.887	7.389%	1600.0	1587.570	1587.818	0.761%
3	800.0	815.649	815.401	1.925%	1600.0	1579.499	1579.251	1.297%
4	800.0	726.232	726.481	9.190%	1600.0	1714.079	1714.327	7.145%
5	800.0	825.474	825.723	3.215%	1600.0	1462.656	1462.905	8.568%

Stage 1 — Stiffness Parameters (k_1, k_2)								
Run	k1 True	k1 Init	k1 Pred	k1 Err%	k2 True	k2 Init	k2 Pred	k2 Err%
1	160000	150273.9	150273.9	6.079%	400000	394441.7	394441.7	1.390%
2	160000	157316.9	157316.9	1.677%	400000	408217.9	408217.9	2.054%
3	160000	144539.8	144539.8	9.663%	400000	407735.0	407735.0	1.934%
4	160000	167888.0	167888.0	4.930%	400000	429114.2	429114.2	7.279%
5	160000	159055.3	159055.3	0.590%	400000	427438.8	427438.8	6.860%

Stage 1 — Damping Coefficients (α, β)								
Run	α True	α Init	α Pred	α Err%	β True	β Init	β Pred	β Err%
1	0.3464	0.31336	0.31495	9.080%	2.904e-05	2.727e-05	2.741e-05	5.610%
2	0.3897	0.36500	0.36684	5.866%	3.267e-05	3.213e-05	3.230e-05	1.146%
3	0.4330	0.46721	0.46487	7.359%	3.630e-05	3.655e-05	3.637e-05	0.188%
4	0.4763	0.48201	0.48436	1.693%	3.993e-05	3.658e-05	3.675e-05	7.959%
5	0.5196	0.48583	0.48829	6.026%	4.356e-05	4.105e-05	4.126e-05	5.279%

Stage 1 — Natural Frequencies & Damping Ratios (ω, ζ)												
Run	ω_1 True	ω_1 Pred	ω_1 Err%	ω_2 True	ω_2 Pred	ω_2 Err%	ζ_1 True	ζ_1 Pred	ζ_1 Err%	ζ_2 True	ζ_2 Pred	ζ_2 Err%
1	10.7200	10.7540	0.317%	20.8590	20.9020	0.206%	0.016312	0.014790	9.329%	0.008606	0.007820	9.132%
2	10.7200	11.0620	3.185%	20.8590	21.1220	1.264%	0.018351	0.016760	8.669%	0.009682	0.009025	6.790%
3	10.7200	10.4720	2.310%	20.8590	20.4280	2.065%	0.020390	0.022385	9.784%	0.010758	0.011750	9.218%
4	10.7200	11.3240	5.632%	20.8590	21.2390	1.826%	0.022429	0.021595	3.720%	0.011834	0.011793	0.347%
5	10.7200	10.8850	1.533%	20.8590	21.7960	4.494%	0.024468	0.022655	7.411%	0.012910	0.011651	9.749%

Figure 6.5: Stage 1 parameter-identification results for all runs

6.2.6 Compact parameter percentage errors for all Stage 1 runs

Run	m_1	m_2	k_1	k_2	α	β	ω_1	ω_2	ζ_1	ζ_2	Mean Err%
1	3.84%	4.68%	6.08%	1.39%	9.08%	5.61%	0.32%	0.21%	9.33%	9.13%	4.97%
2	7.39%	0.76%	1.68%	2.05%	5.87%	1.15%	3.19%	1.26%	8.67%	6.79%	3.88%
3	1.93%	1.30%	9.86%	1.93%	7.36%	0.19%	2.31%	2.06%	9.78%	9.22%	4.57%
4	9.19%	7.15%	4.93%	7.28%	1.69%	7.96%	5.63%	1.83%	3.72%	0.35%	4.97%
5	3.22%	8.57%	0.59%	6.86%	6.03%	5.28%	1.53%	4.49%	7.41%	9.75%	5.37%

Figure 6.6: parameter percentage errors for all Stage 1 runs

Table 6.2.7. Modal property errors for all Stage 1 runs

Run	ω_1 Err%	ω_2 Err%	ζ_1 Err%	ζ_2 Err%
1	0.32%	0.21%	9.33%	9.13%
2	3.19%	1.26%	8.67%	6.79%
3	2.31%	2.06%	9.78%	9.22%
4	5.63%	1.83%	3.72%	0.35%
5	1.53%	4.49%	7.41%	9.75%

Figure 6.7: Modal property errors for all Stage 1 runs

6.2.7 Discussion

The detailed true–initial–predicted table provides a complete numerical picture of Stage 1 parameter identification and should be included directly in the report. It shows that the model generally moved from the initial perturbed values toward physically meaningful predicted values, while keeping the final errors in a moderate range. Modal frequencies were identified more robustly than damping ratios.

6.3 Error Analysis



Figure 6.8: Parameters Within Error Threshold-All Runs

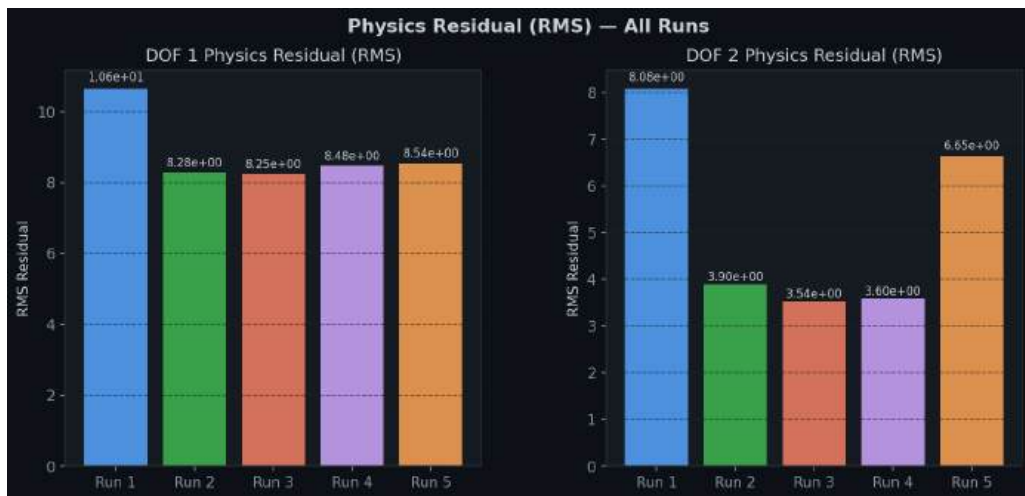


Figure 6.9: Physics Residual (RMS)-All Runs

6.3.1 Parameter accuracy threshold counts for all Stage 1 runs

Run	Params within 5%	Params within 10%	Mean Param Err%
Stage 1 — Parameter Recovery Summary			
1	5 / 10	10 / 10	4.967%
2	6 / 10	10 / 10	3.880%
3	6 / 10	10 / 10	4.574%
4	5 / 10	10 / 10	4.972%
5	4 / 10	10 / 10	5.373%

Figure 6.10: Parameter accuracy threshold counts for all Stage 1 runs

6.3.2 Physics residual RMS for all Stage 1 runs

Run	RMS DOF 1	RMS DOF 2
Stage 1 — Residual RMS per DOF		
1	1.0640e+01	8.0830e+00
2	8.2831e+00	3.8963e+00
3	8.2469e+00	3.5403e+00
4	8.4770e+00	3.5986e+00
5	8.5365e+00	6.6497e+00

Figure 6.11: Physics residual RMS for all Stage 1 runs

6.3.3 Best and worst parameter errors across Stage 1 runs

Stage 1 — Parameter	Parameter Error Range (Min / Maximum Error)	Max across 5 Runs
m_1	1.93%	9.19%
m_2	0.76%	8.57%
k_1	0.59%	9.66%
k_2	1.39%	7.28%
α	1.69%	9.08%
β	0.19%	7.96%
ω_1	0.32%	5.63%
ω_2	0.21%	4.49%
ζ_1	3.72%	9.78%
ζ_2	0.35%	9.75%

■ Low error
 ■ Moderate
 ■ High error

Figure 6.12: Best and worst parameter errors across Stage 1 runs

6.3.4 Discussion

All five runs achieved **10/10 parameters within the 10% threshold**, while **4 to 6 parameters** per run also satisfied the stricter **5% threshold**. The physics residuals remained non-zero but relatively controlled, especially compared with a more unstable identification setting.

6.4 Convergence Behavior



Figure 6.13: Convergence Behavior

6.4.1 Stage 1 training setup

Item	Value
Runs completed	5
Training stage	stage1
Epochs	3000
Warm-up transition visible	Yes
Optimization behavior	Initial rapid drop, jump after warm-up, gradual decrease
Framework role	Baseline PINN parameter identification

6.4.2 Convergence-related summary from final evaluation

Run	Mean NRMSE _{Stage 1}	CoRMS DOF 1 _{Performance}	RMS DOF 2 _{ary}	Mean Param Err%
1	0.28814	1.0640e+01	8.0830e+00	4.967%
2	0.27542	8.2831e+00	3.8963e+00	3.880%
3	0.25778	8.2469e+00	3.5403e+00	4.574%
4	0.25015	8.4770e+00	3.5986e+00	4.972%
5	0.23370	8.5365e+00	6.6497e+00	5.373%

Cell shading: green = better performance, red = worse performance

Figure 6.14: Convergence-related summary from final evaluation stage1

The combined loss history shows the same overall pattern in all runs: early rapid loss reduction, a sharp jump near the warm-up transition, and then a gradual decrease. All runs completed successfully and converged to interpretable parameter solutions.

6.5 Discussion of Performance

Table 6.11. Stage 1 overall performance summary

Metric	Stage 1 — Overall Evaluation Summary	Observation
Runs Evaluated	5	
Mean R^2 Range	−0.0003 to −0.0001	
Mean NRMSE Range	0.23370 to 0.28814	
Mean Parameter Error Range	3.88% to 5.37%	
Parameters within 10%	10 / 10 in all runs	
Parameters within 5%	4 / 10 to 6 / 10	
RMS Residual DOF 1 Range	8.2469e+00 to 1.0640e+01	
RMS Residual DOF 2 Range	3.5403e+00 to 8.0830e+00	

Figure 6.15: overall performance summary stage1

6.5.1 Strong and weak aspects of Stage 1 performance

Aspect	Result
Response reconstruction	Weak
Parameter identification	Moderate and consistent
Modal frequency estimation	Stronger
Damping ratio estimation	More variable
Training stability	Acceptable baseline stability
Physics consistency	Partial, not complete

6.5.2 Discussion

Stage 1 performs better as a **parameter-identification framework** than as a **signal-reconstruction framework**. The detailed identification table in Section 6.2 supports this clearly: the predicted parameters remain physically meaningful and moderately accurate even though the signal metrics are weak.

6.6 Strengths and Limitations of the Proposed Method

6.6.1 Strengths observed in Stage 1

Strength	Stage 1	Key Strengths	Supporting Result
All runs completed successfully		5 / 5 runs completed	
All parameters within 10% in every run		10 / 10 for all runs	
Moderate mean parameter error		3.88% to 5.37%	
Good modal frequency estimation		ω_1 : 0.32%–5.63%, ω_2 : 0.21%–4.49%	
Stable baseline framework		Common convergence pattern across runs	

Figure 6.16: Strengths observed in stage1

6.6.2 Limitations observed in Stage 1

Limitation	Supporting Result
Weak time-domain reconstruction	Mean R^2 near zero in all runs
Moderate signal mismatch remains	Mean NRMSE 0.23370 – 0.28814
Not all parameters within 5%	Only 4 / 10 to 6 / 10 depending on run
Non-zero residuals remain	RMS DOF 1 up to 1.0640e+01, RMS DOF 2 up to 8.083
Uneven difficulty across parameter groups	Damping ratios more variable than modal frequencies

6.6.3 Final Stage 1 interpretation

Stage 1 demonstrates that the proposed PINN framework is feasible for **baseline structural parameter identification**. Its strongest evidence comes from the full true–initial–predicted parameter table, which shows that the model consistently recovers meaningful physical parameters with moderate error. Its main limitation remains weak time-domain response reconstruction and incomplete physics enforcement.

Chapter 7

Results and Discussion

7.1 Overview of Results Stage 2: Fully Blind PINN Framework

Stage 2 results are reported for **5 evaluated runs** under the fully blind PINN setting. The combined training and evaluation summaries are used below.

7.1.1 Predicted vs True Responses

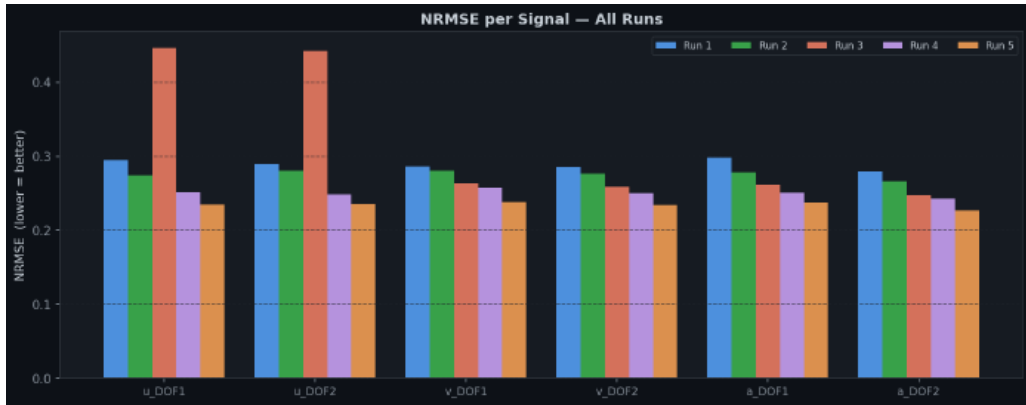


Figure 7.1: NRMSE per All Runs Stage2

7.1.2 Signal reconstruction (R^2) for all Stage 2 run

Run	u_DOF 1	u_DOF 2	v_DOF 1	v_DOF 2	a_DOF 1	a_DOF 2	Mean R^2
1	-0.0015	-0.0192	-0.0000	-0.0000	-0.0003	-0.0003	-0.0036
2	-0.0008	-0.0235	-0.0000	-0.0000	-0.0003	-0.0003	-0.0042
3	-1.8729	-2.0625	-0.0001	-0.0002	-0.0003	-0.0004	-0.6561
4	-0.0004	-0.0042	-0.0001	-0.0001	-0.0003	-0.0003	-0.0009
5	-0.0324	-0.0015	-0.0003	-0.0003	-0.0002	-0.0002	-0.0058

■ Near zero
 ■ Mild
 ■ Moderate
 ■ Large negative

Figure 7.2: Signal Reconstruction (R^2) for all Stage 2 run

7.1.3 Signal reconstruction NRMSE for all Stage 2 runs

Run	u_DOF 1	u_DOF 2	v_DOF 1	v_DOF 2	a_DOF 1	a_DOF 2	Mean NRMSE
1	0.29461	0.28888	0.28601	0.28528	0.29761	0.27936	0.28863
2	0.27423	0.28038	0.28030	0.27630	0.27839	0.26617	0.27596
3	0.44582	0.44236	0.26321	0.25905	0.26094	0.24761	0.31983
4	0.25165	0.24855	0.25685	0.25014	0.25115	0.24304	0.25023
5	0.23454	0.23588	0.23824	0.23371	0.23689	0.22677	0.23434

■ Low
 ■ Mild
 ■ Moderate
 ■ High

Figure 7.3: Signal reconstruction NRMSE for all Stage 2 runs

7.1.4 Discussion

The Stage 2 response reconstruction remained weak overall. Most runs produced mean (R^2) values near zero or slightly negative, while **Run 3** was a clear outlier with a mean (R^2) of **-0.6561**, driven by the very poor displacement reconstruction in both DOFs. The mean NRMSE values ranged from **0.23434** in Run 5 to **0.31983** in Run 3. Thus, even in the blind setting, the framework did not achieve strong time-domain agreement between predicted and measured responses.

7.2 Estimated Structural Parameters

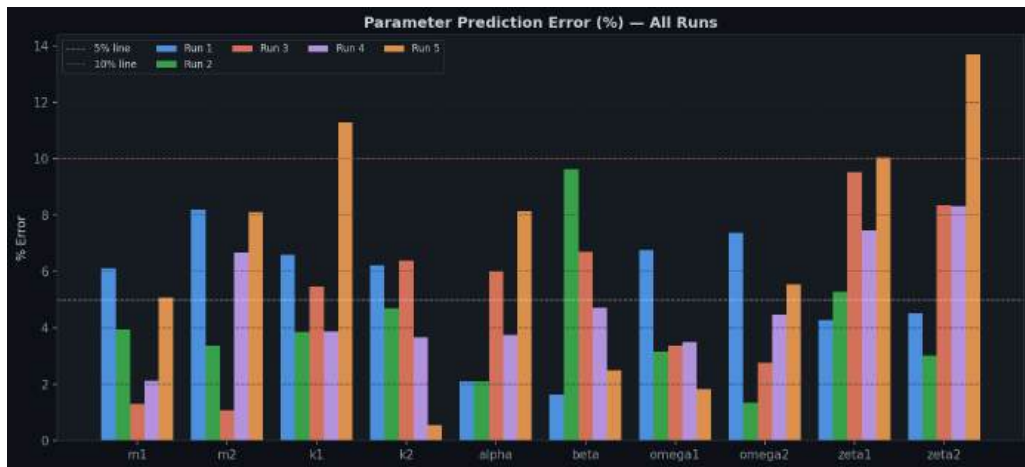


Figure 7.4: Parameter % Error All Stage 2 runs

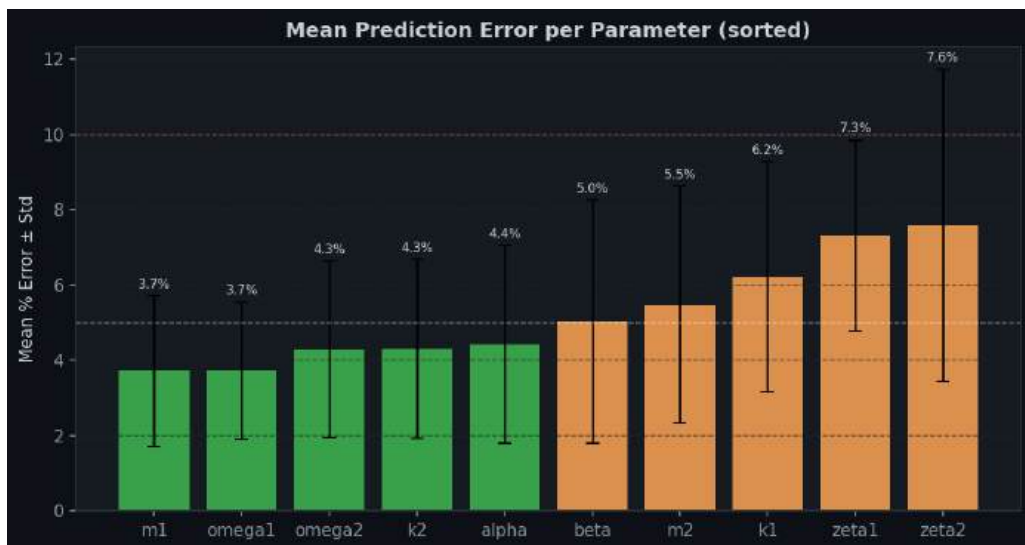


Figure 7.5: Mean Prediction Error per Parameter Stage 2

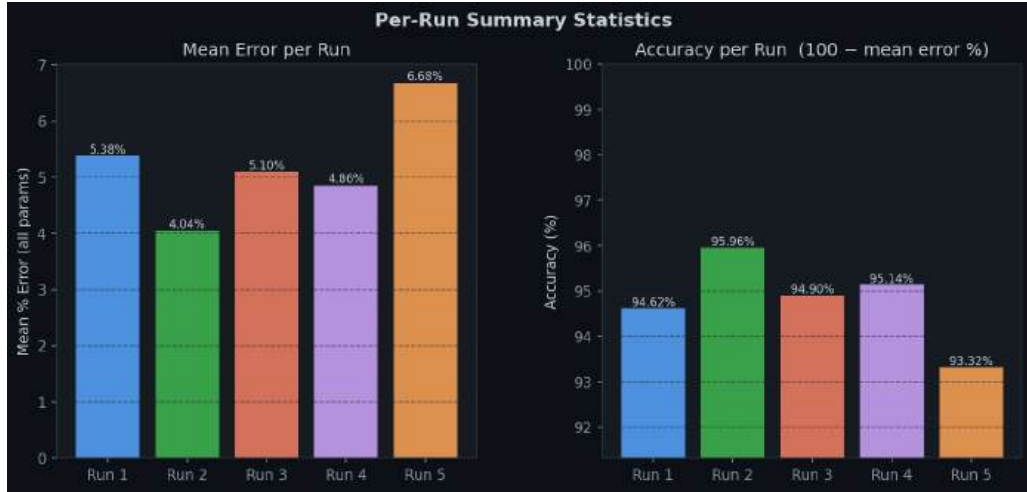


Figure 7.6: Per- Run Summary Stage 2

7.2.1 Parameter-level summary across all Stage 2 runs

Stage 2 – Mass Parameters (m_1 , m_2)								
Run	m1 True	m1 Init	m1 Pred	m1 Err%	m2 True	m2 Init	m2 Pred	m2 Err%
1	800.0	746.740	751.087	6.114%	1600.0	1465.919	1468.810	8.199%
2	800.0	824.722	831.593	3.949%	1600.0	1535.131	1545.982	3.376%
3	800.0	798.766	810.466	1.308%	1600.0	1582.590	1582.843	1.072%
4	800.0	814.040	817.133	2.142%	1600.0	1697.420	1706.919	6.682%
5	800.0	748.891	759.299	5.088%	1600.0	1738.127	1729.616	8.101%

Figure 7.7: parameter-identification results for Mass Stage 2

Stage 2 – Stiffness Parameters (k_1 , k_2)								
Run	k1 True	k1 Init	k1 Pred	k1 Err%	k2 True	k2 Init	k2 Pred	k2 Err%
1	160000	174727.2	170553.4	6.596%	400000	435076.0	424876.1	6.219%
2	160000	160764.4	153824.8	3.859%	400000	401154.6	381214.1	4.696%
3	160000	156964.1	151244.2	5.472%	400000	387711.5	374449.4	6.388%
4	160000	159050.7	153805.5	3.872%	400000	398855.7	385358.5	3.660%
5	160000	149081.0	141937.2	11.289%	400000	420116.1	397789.3	0.553%

Figure 7.8: parameter-identification results for Stiffness Stage 2

Stage 2 — Damping Coefficients (α , β)								
Run	α True	α Init	α Pred	α Err%	β True	β Init	β Pred	β Err%
1	0.3464	0.35477	0.35369	2.105%	2.904e-05	2.861e-05	2.856e-05	1.650%
2	0.3897	0.40327	0.39793	2.111%	3.267e-05	3.007e-05	2.953e-05	9.612%
3	0.4330	0.46306	0.45901	6.008%	3.630e-05	3.404e-05	3.387e-05	6.687%
4	0.4763	0.50095	0.49420	3.759%	3.993e-05	4.226e-05	4.181e-05	4.718%
5	0.5196	0.55461	0.56195	8.151%	4.356e-05	4.201e-05	4.247e-05	2.497%

Figure 7.9: parameter-identification results for Damping Stage 2

Stage 2 — Natural Frequencies & Damping Ratios (ω , ζ)												
Run	ω_1 True	ω_1 Pred	ω_1 Err%	ω_2 True	ω_2 Pred	ω_2 Err%	ζ_1 True	ζ_1 Pred	ζ_1 Err%	ζ_2 True	ζ_2 Pred	ζ_2 Err%
1	10.7202	11.4448	6.760%	20.8585	22.3936	7.359%	0.016312	0.015615	4.271%	0.008806	0.008217	4.526%
2	10.7202	10.3803	3.170%	20.8585	20.5745	1.362%	0.018351	0.019321	5.283%	0.009682	0.009974	3.015%
3	10.7202	10.3586	3.373%	20.8585	20.2838	2.755%	0.020390	0.021332	9.522%	0.010758	0.011658	8.368%
4	10.7202	10.3449	3.500%	20.8585	19.9268	4.467%	0.022429	0.024103	7.461%	0.011834	0.012817	8.309%
5	10.7202	10.5230	1.839%	20.8585	19.7040	5.535%	0.024468	0.026925	10.039%	0.012910	0.014678	13.700%

Figure 7.10: parameter-identification results for Natural Frequencies & Damping Ratios Stage 2

7.2.2 Parameter-level summary across all Stage 2 runs

Stage 2 — Parameter Error Statistics & R ² Summary					
Param	Mean Err%	Std%	Min%	Max%	R ²
m_1	3.720%	1.787%	1.308%	6.114%	-1e15
m_2	5.486%	2.813%	1.072%	8.199%	-1e16
k_1	6.218%	2.738%	3.859%	11.289%	-1e20
k_2	4.303%	2.128%	0.553%	6.388%	-1e21
α	4.427%	2.348%	2.105%	8.151%	0.84470
β	5.033%	2.887%	1.650%	9.612%	0.84397
ω_1	3.728%	1.628%	1.839%	6.760%	-1e11
ω_2	4.296%	2.094%	1.362%	7.359%	-1e12
ζ_1	7.315%	2.268%	4.271%	10.039%	0.66256
ζ_2	7.584%	3.709%	3.015%	13.700%	0.55569

■ Low / Good R²
■ Moderate
 ■ High / Bad R²

Figure 7.11: Parameter-level summary across all Stage 2

7.2.3 Per-run Stage 2 parameter-identification errors

Stage 2 — Per-Run Parameter Errors & Accuracy												
Run	m_1	m_2	k_1	k_2	α	β	ω_1	ω_2	ζ_1	ζ_2	Mean Err%	Accuracy%
1	6.11%	8.20%	6.60%	6.22%	2.10%	1.65%	6.76%	7.36%	4.27%	4.53%	5.38%	94.62%
2	3.95%	3.38%	3.86%	4.70%	2.11%	9.61%	3.17%	1.36%	5.28%	3.02%	4.04%	95.96%
3	1.31%	1.07%	5.47%	6.39%	6.01%	6.69%	3.37%	2.76%	9.52%	8.37%	5.10%	94.90%
4	2.14%	6.68%	3.87%	3.66%	3.76%	4.72%	3.50%	4.47%	7.46%	8.31%	4.86%	95.14%
5	5.09%	8.10%	11.29%	0.55%	8.15%	2.50%	1.84%	5.53%	10.04%	13.70%	6.68%	93.32%

■ $\leq 3\%$ / $\geq 96\%$ (Good)
 ■ 3-7% / 94-96% (Moderate)
 ■ $> 7\%$ / $< 94\%$ (High)

Figure 7.12: Per- Run Parameter Error & Accuracy Stage 2

7.2.4 Compact interpretation of Stage 2 parameter difficulty

Stage 2 — Parameter Difficulty Classification			
Easier / Stronger Parameters		Harder / Weaker Parameters	
Parameter	Mean Err%	Parameter	Mean Err%
m_1	3.720%	ζ_2	7.584%
ω_1	3.728%	ζ_1	7.315%
k_2	4.303%	k_1	6.218%
ω_2	4.296%	m_2	5.486%
α	4.427%		
β	5.033%		

Figure 7.13: Per- Run Parameter Difficulty Classification Stage 2

7.2.5 Discussion

The full detailed table should be included directly in the report because it shows the entire identification path from **true value** to **initial blind guess** to **predicted value**, together with the final percentage error. This makes the Stage 2 parameter-identification section complete and transparent. The summary tables then help condense the main trends. Overall, Stage 2 retained moderate parameter-estimation capability despite the blind setting, but with greater spread than Stage 1. The strongest average estimates were obtained for **m1**, **omega1**, **omega2**, **k2**, and **alpha**, while the damping ratios remained the hardest quantities.

7.3 Error Analysis

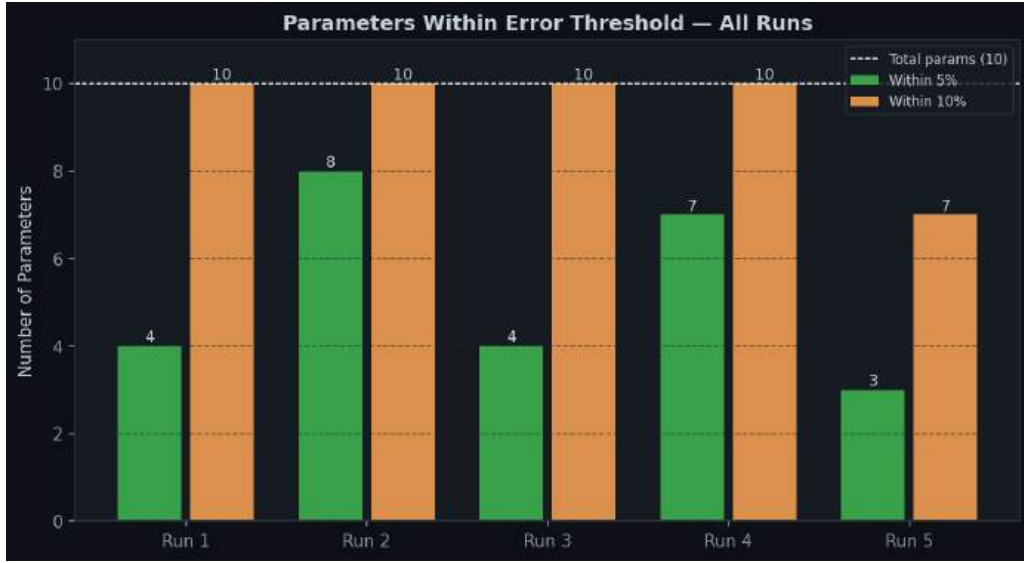


Figure 7.14: Parameters Within Error Threshold-All Runs Stage 2

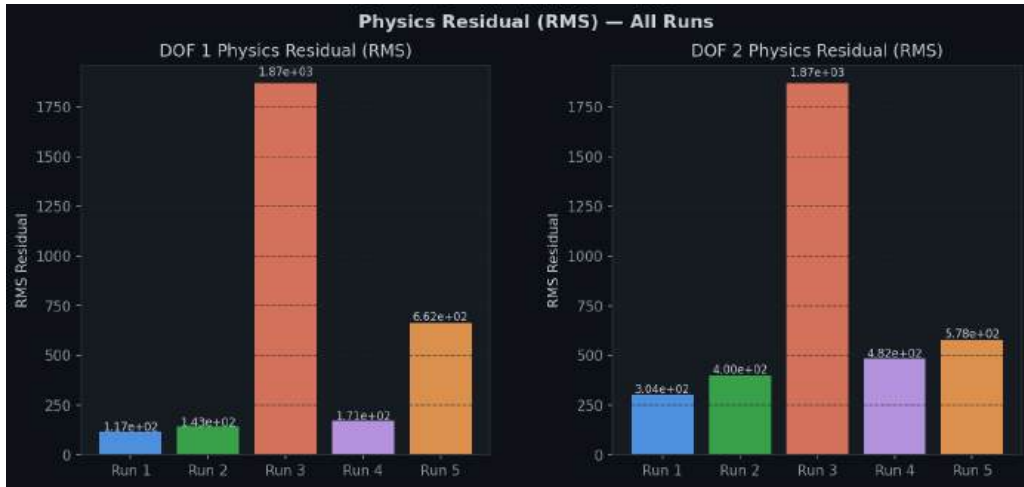


Figure 7.15: Physics Residual (RMS)-All Runs Stage 2

7.3.1 Parameter accuracy threshold counts for all Stage 2 runs

Run	Params within 5%	Params within 10%	Mean Param Err%
1	4 / 10	10 / 10	5.380%
2	8 / 10	10 / 10	4.043%
3	4 / 10	10 / 10	5.095%
4	7 / 10	10 / 10	4.857%
5	3 / 10	7 / 10	6.679%

7.3.2 Physics residual RMS for Stage 2

Run	RMS DOF 1	RMS DOF 2
1	1.1687e+02	3.0424e+02
2	1.4287e+02	3.9997e+02
3	1.8671e+03	1.8674e+03
4	1.7084e+02	4.8236e+02
5	6.6219e+02	5.7782e+02

7.3.3 Best and worst Stage 2 cases

Metric	Best Run	Best Value	Worst Run	Worst Value
Mean R^2	Run 4	−0.0009	Run 3	−0.6561
Mean NRMSE	Run 5	0.23434	Run 3	0.31983
Mean Param Err%	Run 2	4.043%	Run 5	6.679%

7.3.4 Discussion

The threshold analysis shows that Runs 1–4 still kept **all 10 parameters within 10%**, but Run 5 dropped to **7/10**, with only **3/10** parameters within the stricter **5%** range. The residual analysis shows much larger RMS values than those seen in Stage 1, especially in Run 3 and Run 5, confirming that the blind formulation increased instability and made consistent convergence harder.

7.4 Convergence Behavior

Convergence is summarized using the training configuration and the final residual/performance spread.

Table 6.24. Stage 2 training configuration snapshot

Item	Value
Runs Completed	5
Epochs (Adam)	4000
Warm-up Epochs	800
Ramp Epochs	1600
LBFGS Refinement	True
LBFGS Max Iterations	600
LBFGS Learning Rate	0.5
Gradient Clipping	5.0
Device	cuda
Seed Base	2000

7.4.1 Loss and optimization settings for Stage 2

Category	Setting	Value
Data	n_colloc_factor	4.0
Loss	w_u	2.0
Loss	w_v	0.5
Loss	w_a	1.5
Loss	w_phys	2.0
Loss	w_modal	0.0
Loss	w_zeta	0.0
Loss	w_param	0.0
Loss	lambda_scale_prior	0.005
Loss	perturb_level	0.1
Opt	lr_nn	0.001
Opt	lr_k	0.001
Opt	lr_m	0.0005
Opt	lr_damp	0.0005
Sched	factor	0.5
Sched	patience	300

7.4.2 Convergence-related numerical summary for Stage 2

Run	Mean NRMSE	Mean Param Err%	RMS DOF 1	RMS DOF 2	Accuracy%
1	0.28863	5.380%	1.1687e+02	3.0424e+02	94.62%
2	0.27596	4.043%	1.4287e+02	3.9997e+02	95.96%
3	0.31983	5.095%	1.8671e+03	1.8674e+03	94.90%
4	0.25023	4.857%	1.7084e+02	4.8236e+02	95.14%
5	0.23434	6.679%	6.6219e+02	5.7782e+02	93.32%

7.4.3 Discussion

Stage 2 used a richer optimization setup than Stage 1, including longer Adam training, warm-up, ramped physics activation, gradient clipping, and LBFGS refinement. Even though all runs completed successfully, the final residual spread shows that convergence quality was uneven. Runs 3 and 5 were especially unstable, as reflected by their higher residual magnitudes and weaker overall performance.

7.5 Discussion of Performance

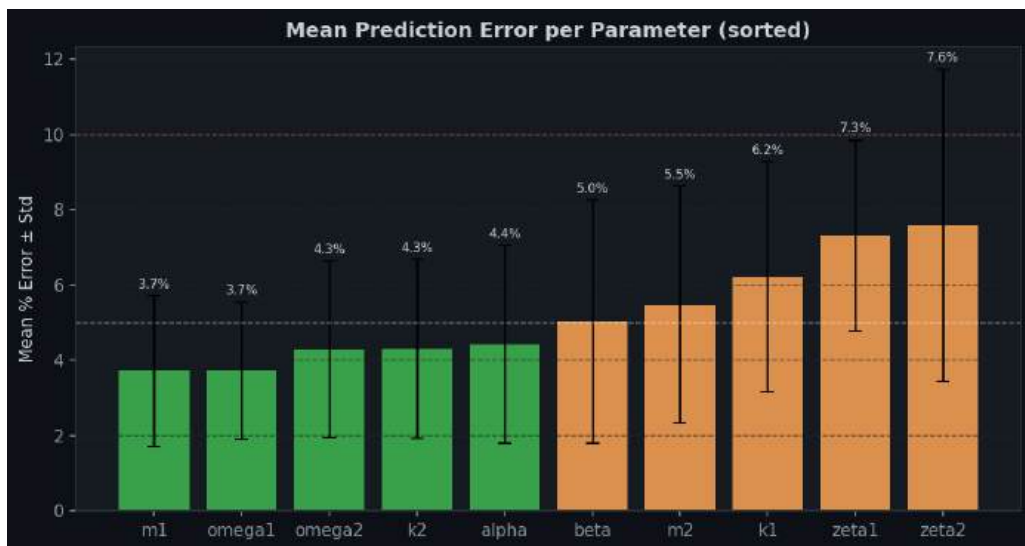


Figure 7.16: Mean Prediction Error per Parameter Stage 2

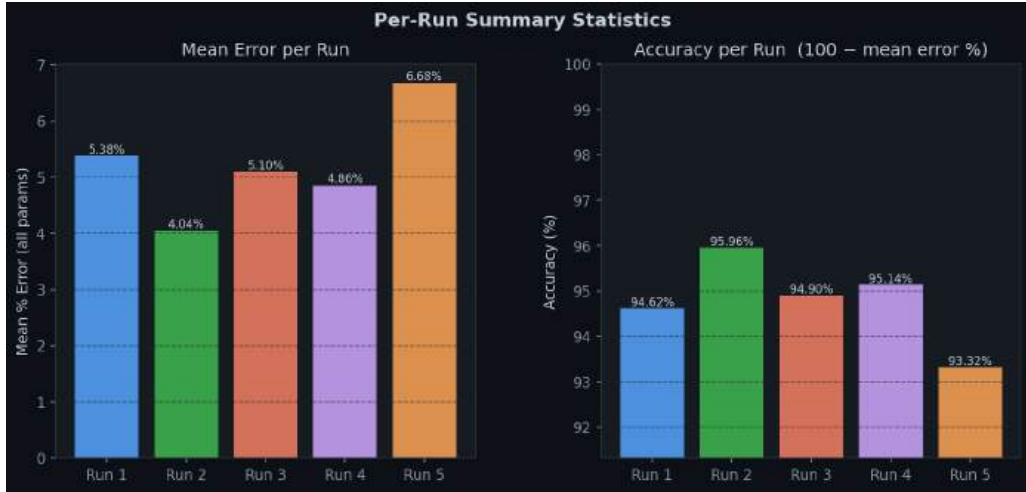


Figure 7.17: Per-Run Summary Statistics Stage 2

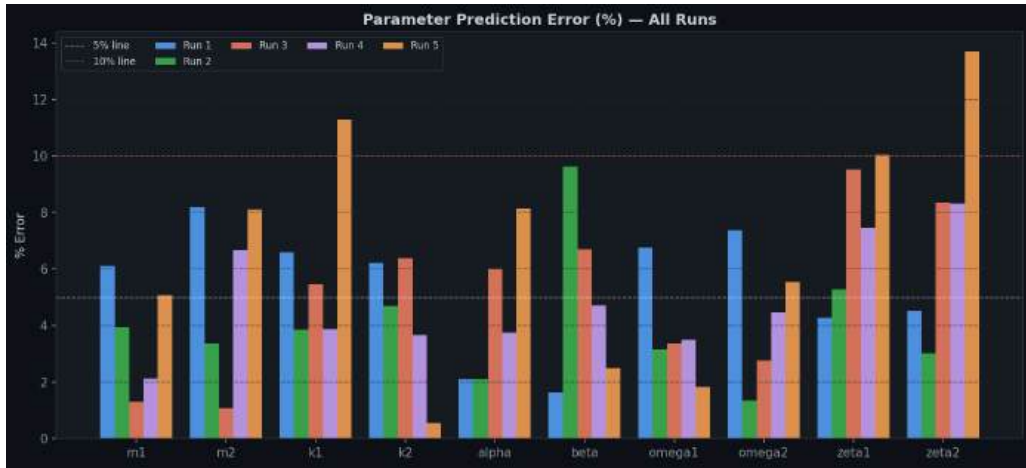


Figure 7.18: Parameter Prediction Error % Stage 2

7.5.1 Overall Stage 2 performance summary

Metric	Observation
Runs Evaluated	5
Mean R^2 Range	−0.6561 to −0.0009
Mean NRMSE Range	0.23434 to 0.31983
Mean Parameter Error Range	4.043% to 6.679%
Parameters within 10%	10/10 in Runs 1–4 · 7/10 in Run 5
Parameters within 5%	3/10 to 8/10
RMS Residual DOF 1 Range	1.1687e+02 to 1.8671e+03
RMS Residual DOF 2 Range	3.0424e+02 to 1.8674e+03
Best Overall Run	Run 2
Weakest Overall Run	Run 5

7.5.2 Strong and weak aspects of Stage 2 performance

Aspect	Result
Response Reconstruction	Weak and variable
Parameter Identification	Moderate, but less consistent than Stage 1
Best Run	Run 2
Worst Signal Reconstruction	Run 3
Weakest Parameter Stability	Run 5
Strongest Parameter Groups	$m_1, \omega_1, \omega_2, k_2, \alpha$
Weakest Parameter Groups	ζ_1, ζ_2, k_1

7.5.3 Discussion

Stage 2 remained feasible as a blind parameter-identification framework, but its performance was less stable than Stage 1. Signal reconstruction was still weak, and the blind setting introduced much stronger run-to-run variability. Nevertheless, the model still recovered many structural quantities with moderate accuracy, confirming that blind physics-informed identification is possible even in a more difficult setting.

7.6 Strengths and Limitations of the Proposed Method

7.6.1 Strengths observed in Stage 2

Strength	Supporting Result
Blind identification remained feasible	Moderate parameter accuracy in all runs
Strongest run performed well	Run 2: mean error 4.043%, 8/10 within 5%
Several parameters below $\sim 5\%$ mean error	$m_1, k_2, \alpha, \omega_1, \omega_2$
All runs completed successfully	5 / 5 runs completed
Fully blind formulation achieved interpretable solutions	Parameter sets remained physically meaningful

7.6.2 Limitations observed in Stage 2

Limitation	Supporting Result
Weak time-domain reconstruction	Mean R^2 near zero or negative in all runs
Severe outlier behaviour in one run	Run 3 mean $R^2 = -0.6561$
Larger residuals than Stage 1	RMS up to $1.8674\text{e}+03$
Strong run-to-run variability	Run 2 = 4.043% mean err · Run 5 = 6.679%
Damping ratios remained hardest outputs	ζ_1 mean = 7.315%, ζ_2 mean = 7.584%
Not all runs preserved 10/10 within 10%	Run 5 dropped to 7/10

7.6.3 Final Stage 2 interpretation

Stage 2 demonstrates that **fully blind physics-informed structural parameter identification is feasible**, but also shows that the increased realism of the formulation comes with noticeably greater instability. The framework remained capable of recovering useful structural parameters, but signal reconstruction stayed weak and the residual magnitudes became substantially larger, especially in difficult runs.

Chapter 8

Conclusion and Future Work

8.1 Summary of Major Findings

This study investigated the use of **Physics-Informed Neural Networks (PINNs)** for **multi-degree-of-freedom** structural system modelling and **parameter identification** in the context of **Structural Health Monitoring (SHM)**. The work focused on a **2DOF** structural dynamic system described by the governing equation of motion

$$M\ddot{u}(t) + C\dot{u}(t) + Ku(t) = F(t)$$

where the objective was to estimate the unknown structural parameters from simulated dynamic response data while preserving physical consistency. The overall project was organized into two methodological stages: Stage 1, which served as a baseline PINN framework, and Stage 2, which extended the formulation into a more difficult fully blind inverse-identification setting. Both stages used the same synthetic dataset of 216 simulation runs generated from a 2DOF mass-spring-damper system. The dataset included time-series signals for displacement, velocity, acceleration, and external forces, together with the true structural parameters and modal properties of each run.

The results showed that the proposed framework was able to recover physically meaningful structural parameters with moderate accuracy, especially for modal frequencies and several physical coefficients. In **Stage 1**, the framework achieved stable baseline performance across the five evaluated runs. Although the signal reconstruction remained weak in terms of (R^2), the estimated structural parameters were generally within acceptable error bounds, and all ten parameters were recovered within 10% error for all evaluated runs. This confirmed that the baseline PINN formulation was effective as a first-step inverse-identification framework.

In **Stage 2**, the problem was made more challenging by removing the mild supervision present in the baseline setting and enforcing a stricter blind physics-informed learning process. Despite the increased difficulty, the framework still produced interpretable pa-

parameter estimates and remained capable of recovering several structural quantities with moderate accuracy. However, the results also showed larger residuals, greater run-to-run variation, and weaker signal reconstruction compared to Stage 1. These findings demonstrate that blind PINN-based system identification is feasible, but also substantially more difficult and less stable than the baseline setting.

Overall, the main findings of the study can be summarized as follows:

1. A PINN-based framework can be successfully formulated for 2DOF structural parameter identification.
2. The governing structural dynamics equations can be embedded directly into the neural-network training process through a physics-based residual formulation.
3. The proposed method is more effective at **recovering physical parameters** than at **accurately reproducing complete time-domain response histories**.
4. The baseline Stage 1 framework provides stable and interpretable inverse-identification results.
5. The fully blind Stage 2 formulation is feasible, but introduces significantly greater instability and sensitivity.

These findings confirm that PINNs provide a meaningful and physically grounded computational framework for SHM-oriented inverse modelling problems.

8.2 Overall Performance of the Proposed PINN Framework

The overall performance of the proposed PINN framework can be evaluated from two perspectives: **response reconstruction performance** and **structural parameter identification performance**.

From the response-reconstruction perspective, both stages showed limited success. In most evaluated runs, the signal-level (R^2) values remained very close to zero or slightly negative, which indicates that the predicted displacement, velocity, and acceleration responses did not match the measured time histories strongly in the statistical sense. Even though the normalized root mean squared error (NRMSE) showed some gradual improvement across several runs, the framework did not achieve high-fidelity signal reconstruction in either stage.

From the parameter-identification perspective, however, the framework performed much better. The PINN model was able to estimate masses, stiffnesses, Rayleigh damping

coefficients, natural frequencies, and damping ratios with moderate accuracy. In Stage 1, the mean parameter errors remained in a relatively controlled range, and all evaluated runs achieved full 10/10 parameter recovery within the broader 10% error threshold. This demonstrates that the model captured important structural information even when the predicted time-series responses themselves were not strongly aligned with the measured signals.

Stage 2 maintained this capability to some extent, but with lower consistency. Several parameters, particularly $m_1, k_2, \omega_1, \omega_2$, and α , were still estimated with moderate mean accuracy. However, damping ratios and certain stiffness or mass terms became more difficult to recover under the fully blind setting. Furthermore, the physics residuals in Stage 2 became substantially larger than those in Stage 1, which indicates that strict blind identification placed far more pressure on the optimization process.

Therefore, the overall performance of the proposed PINN framework may be summarized as follows:

- It is **effective** as a **physically informed parameter-identification framework**.
- It is **less effective** as a **direct signal-reconstruction framework**.
- It **performs better in the baseline formulation** than in the fully blind inverse-identification setting.
- Its strongest outputs are generally **frequency-related quantities** and some physical coefficients.
- Its weakest outputs are generally **damping ratios** and some runs with poor convergence.

Thus, the proposed PINN framework should be viewed primarily as a **physics-based inverse modelling tool** for structural parameter recovery rather than as a purely response-fitting model.

8.3 Limitations of the Present Study

Although the study successfully demonstrated the feasibility of PINNs for structural parameter identification, several limitations remain.

The first limitation is the use of a **fully synthetic dataset**. While simulated data provides complete access to the true structural parameters and enables controlled evaluation, it does not fully reflect the complexity of real-world SHM environments. Real sensor

data often includes more complex noise, missing values, modelling uncertainty, changing environmental conditions, and imperfect force measurements. Therefore, the conclusions of this study are currently limited to a simulation-based setting.

The second limitation is the restriction to a **2DOF structural system**. Although the 2DOF model is an appropriate and meaningful first step for developing and validating the methodology, real engineering structures often have many more degrees of freedom and more complex modal interactions. Thus, the present study does not yet demonstrate scalability to larger and more realistic structural systems.

The third limitation concerns the **weak signal reconstruction quality**. In both stages, the PINN framework showed limited ability to reproduce the measured displacement, velocity, and acceleration histories accurately in the time domain. This means that the model captured structural parameters more effectively than it captured the full waveform behaviour.

The fourth limitation is the **optimization difficulty** associated with PINNs. The training process involves balancing data loss, physics loss, and parameter estimation simultaneously. In Stage 2 especially, the convergence became more unstable, and some runs showed significantly higher residual levels than others. This demonstrates that the method is sensitive to initialization, loss weighting, and optimization strategy.

The fifth limitation is that the study mainly focused on **parameter identification**, not on actual damage scenarios such as stiffness reduction due to cracks or time-varying degradation. Although the framework is relevant for SHM, direct damage classification or localization was not explicitly implemented in this project.

A further limitation is that the present study did not include a full experimental comparison with **real measured data**, nor did it include deployment considerations such as online monitoring, real-time inference, uncertainty quantification, or sensor placement optimization.

Therefore, while the study provides a strong methodological contribution, it should still be interpreted as a controlled computational investigation rather than a final practical SHM solution.

8.4 Comparison Between Data-Driven Models and PINNs

An important conclusion of this work is the difference between **purely data-driven models** and **Physics-Informed Neural Networks** for structural system identification.

A data-driven model learns the mapping from input signals to output parameters entirely from data. Its strength lies in its flexibility and ability to fit complex patterns when a large amount of training data is available. In many cases, data-driven models can produce good prediction accuracy when trained on representative datasets. However, they have important limitations. They depend strongly on data quality and quantity,

may generalize poorly outside the training distribution, and can produce outputs that violate the governing physical laws of the system.

In contrast, a PINN does not rely on data alone. Instead, it incorporates the governing structural dynamics equations directly into the training process. This gives the model an important advantage: the predictions are guided not only by observations, but also by the known physics of the system. As a result, PINNs are especially attractive for inverse problems in structural dynamics, where data may be limited but the governing equations are known.

Based on the present study, the comparison can be summarized as follows:

Aspect	Data-Driven Models	Physics-Informed NNs
Learning Source	Data only	Data + governing physical laws
Need for Large Datasets	High	Lower than purely data-driven
Physical Consistency	Not guaranteed	Explicitly enforced via physics loss
Generalisation Outside Training	Can be weak	Potentially stronger if physics well enforced
Interpretability	Lower	Higher due to physical constraints
Optimisation Difficulty	Usually simpler	More difficult and sensitive
Suitability for Inverse Struct. Dynamics	Good if enough data exists	Very suitable, esp. when physics is known
Signal Fitting Ability	Often strong	Not always strong in practice
Parameter Identification	Possible	Strong advantage due to physical embedding

In the context of this project, the PINN framework showed a clear strength in **recovering physically meaningful structural parameters**, even when full signal reconstruction remained weak. A purely data-driven model may in some cases fit the response signals better, but it would not automatically guarantee physical consistency. Therefore, the PINN approach is particularly valuable when the objective is not only prediction accuracy, but also **physically consistent structural parameter estimation**.

Thus, the comparison suggests that data-driven models and PINNs should not be

seen as direct competitors only. Instead, they can be understood as complementary approaches. Data-driven models may be better suited for pure predictive performance when sufficient data is available, whereas PINNs are especially valuable when physical consistency, inverse modelling, and parameter interpretability are important.

8.5 Suggested Improvements

Based on the observed limitations and performance patterns, several improvements can be suggested for the proposed framework.

The first improvement concerns **loss-function balancing**. Since the model must satisfy both data fitting and physics consistency, the relative weighting between these terms plays a crucial role. More systematic tuning of loss weights, adaptive weighting strategies, or curriculum-based activation of physics terms could improve training stability and final performance.

The second improvement is the use of **better optimization strategies**. Although the framework already used a combination of Adam-based training and LBFGS refinement, the convergence remained unstable in some blind runs. More advanced optimizers, better warm-up strategies, parameter-specific learning-rate schedules, and improved initialization procedures may help reduce the large residual spikes observed in some Stage 2 runs.

A third improvement is the inclusion of **uncertainty estimation**. In structural health monitoring applications, it is often important not only to predict a parameter value, but also to estimate the confidence or uncertainty associated with it. Bayesian PINNs, ensemble methods, or probabilistic formulations could make the framework more useful for practical SHM decisions.

A fourth improvement is to include **more realistic noise modelling**. In the current study, the synthetic dataset is well controlled and only limited measurement noise is considered. Stronger and more diverse noise scenarios would provide a better test of model robustness.

A fifth improvement is the extension to **hybrid modelling approaches**. Since data-driven models may offer stronger signal-fitting ability while PINNs offer better physical consistency, a hybrid model combining both ideas could potentially improve overall performance.

A sixth improvement is to extend the framework from parameter estimation to **explicit damage indicators**, such as stiffness-loss localization, damage severity estimation, or time-varying structural degradation tracking.

Finally, improvements in **visualization, evaluation metrics, and model selection criteria** could also support better interpretation of results and clearer comparison across runs and stages.

8.6 Future Research Directions

This study opens several promising directions for future research.

The most immediate next step is to extend the framework from a **2DOF system to higher-dimensional MDOF systems**. This would test the scalability of the approach and make the methodology more relevant to real engineering structures.

A second research direction is the application of the proposed framework to **real experimental or field-measured SHM datasets**. This is essential for validating whether the model can operate effectively under realistic noise, uncertainty, and sensor limitations.

A third direction is the development of **damage-sensitive PINN formulations**, where structural damage is represented explicitly through local stiffness changes, time-varying parameters, or degradation variables. This would bring the framework closer to direct SHM applications such as damage detection, localization, and severity estimation.

A fourth direction is the exploration of **hybrid data-driven and physics-informed models**. Such models could combine the predictive flexibility of data-driven architectures with the physical consistency of PINNs.

A fifth direction is the incorporation of **uncertainty quantification and probabilistic inference**, which would allow the framework to provide confidence-aware structural parameter estimates.

A sixth research direction is the design of **online or real-time SHM systems**, where PINNs are adapted for streaming data and continuous structural state updating.

Finally, future work could investigate **more advanced PINN architectures**, improved residual formulations, domain decomposition, adaptive collocation strategies, and better physics regularization methods to improve convergence and accuracy in blind inverse problems.

In summary, this study provides a strong methodological foundation for future SHM-oriented inverse modelling research. While the current framework is still limited in several respects, it demonstrates the clear potential of physics-informed learning as an important direction for structural parameter estimation and structural health monitoring.